

Architetture Multi-Agente per la Generazione e Validazione Formale della Documentazione del Software C/C++

Nicola Flego

Anno Accademico 2025/2026

Indice

1	Introduzione	5
1.1	Il Paradigma dei Large Language Model nell'Ingegneria del Software	5
1.2	Il Ruolo Critico della Documentazione del Software: Dalla Manutenzione al "Continuous Drift"	7
1.3	Evoluzione Storica del Code Summarization: Le Quattro Epoche	8
1.3.1	L'Era delle Euristiche e dei Template (pre-2010)	8
1.3.2	L'Era dell'Information Retrieval (primi anni 2010)	9
1.3.3	L'Era del Deep Learning Iniziale (metà anni 2010)	9
1.3.4	Il "Soffitto Tecnologico" Pre-LLM (The Deep Learning Ceiling)	10
1.4	Dall'Analisi Atomica alla Scala Repository: Micro-Scale vs Macro-Scale	11
1.4.1	Micro-Scale: Dalla Funzione alla Documentazione di Dettaglio	11
1.4.2	Macro-Scale: Dalla Codebase alla Mappatura Architeturale	12
1.5	Stato dell'Arte dei Framework di Repository: CodeWiki, RepoAgent e DocAgent	13
1.5.1	CodeWiki (Hoang et al., 2026)	13
1.5.2	RepoAgent (Luo et al., 2024)	13
1.5.3	DocAgent (Yang et al., 2025)	14
1.5.4	RepoSummary (Zhu et al., 2025)	14
1.5.5	Quadro Comparativo Sinottico	15
1.6	Obiettivo della Tesi e "Strategic Blueprint" Proposto	15
1.6.1	Definizione dell'Obiettivo Primario	15
1.6.2	L'Architettura Modulare a Quattro Livelli (Strategic Blueprint)	15
1.6.3	I Tre Principi Cardine della Tesi	17
1.6.4	Struttura e Organizzazione della Tesi	18
2	Tassonomia e Formulazione Analitica delle Metriche di Valutazione	21
2.1	Introduzione e Motivazione	21
2.2	Stato dell'Arte e Tassonomia a Quattro Livelli	22
2.3	Descrizione Analitica delle Metriche di Benchmark	23
2.3.1	Livello I: Metriche Formali e Contratti AST	24
2.3.2	Livello II: Metriche di Qualità Software e Actionability	26
2.3.3	Livello III: Metriche Semantiche e Natural Language Processing	29

2.3.4	Livello III (Avanzato): Framework LLM-as-a-Judge con Campionamento Monte Carlo	32
2.3.5	Livello IV: Validazione Downstream e Testing Comportamentale	34
3	Validazione Sperimentale e Analisi dei Risultati	37
3.1	Setup Sperimentale e Metodologia di Benchmark	37
3.1.1	Strategie di Campionamento e Controllo Sperimentale . . .	38
3.2	Analisi Comparativa delle Rappresentazioni (Cross-Modal)	38
3.2.1	Analisi di CodeBERT vs SBERT/BERTScore: Resilienza Sintattica vs Cecità Semantica	39
3.3	Valutazione Sperimentale ad Armi Pari delle Metriche NLP	40
3.3.1	Analisi Dettagliata per Singola Metrica NLP	42
3.4	Validazione Empirica delle Metriche Avanzate di Livello II, III e IV	48
3.4.1	Concept Checklist Score	48
3.4.2	LLM-as-a-Judge: Disaccoppiamento tra Faithfulness e Alignment	50
3.4.3	Round-Trip Differential Testing: Validazione Comportamentale a Scatola Nera	51
3.4.4	Downstream Code Retrieval MRR	51
3.5	Studi Critici Avanzati: Stress Test, Camouflage e Finestre di Contesto	52
3.5.1	Confronto delle Metriche sotto Stress Test Avversari	52
3.5.2	Distrazione da Fluff e Camuffamento di Allucinazioni nel Giudice LLM	54
3.5.3	Studio sui Limiti di Troncamento Contestuale e Sensibilità alla Posizione	54
3.6	Risultati Empirici dell'Ultimo Benchmark Reale (Multi-Agent) . .	56
3.6.1	Analisi Disaggregata per Tassonomia Funzionale a 3 Classi	56
3.6.2	Diagnostica Causale degli Errori nel Round-Trip	58
3.6.3	Residui della Valutazione: Parafrasi Robusta vs Allucinazione Plausibile	58
3.7	Conclusioni e Matrice di Applicabilità delle Metriche	60

Capitolo 1

Introduzione

1.1 Il Paradigma dei Large Language Model nell'Ingegneria del Software

Negli ultimi anni, la disciplina dell'ingegneria del software (Software Engineering, SE) ha assistito a una profonda transizione di paradigma, trainata dall'avvento e dalla rapida maturazione dei modelli linguistici a grande scala (Large Language Model, LLM) fondazionali. Inizialmente concepiti e ottimizzati nell'alveo dell'Elaborazione del Linguaggio Naturale (Natural Language Processing, NLP) per compiti generici di modellazione sequenziale non supervisionata, tali architetture neurali — prevalentemente basate sul modello *Transformer* con meccanismo di auto-attenzione stocastica (*self-attention*) e decodifica causale autoregressiva — hanno progressivamente trasceso il dominio del testo naturale.

La convergenza tra l'enorme disponibilità di dati archiviati nelle piattaforme di social coding e lo scaling computazionale ha favorito la nascita di modelli esplicitamente addestrati o rifiniti su vasti corpora di codice sorgente bilingue (codice e annotazioni correlate). Sistemi pionieristici quali CodeX [1, 2], StarCoder [3, 4], CodeGen [5] e PolyCoder [6] hanno dimostrato che il codice sorgente, pur possedendo proprietà sintattico-formali e vincoli deterministici di compilazione assenti nel linguaggio naturale umano, manifesta una forte “ipotesi di naturalezza” (*naturalness hypothesis*): i programmi scritti da esseri umani esibiscono una ridondanza statistica, una regolarità lessicale e pattern idiomatici che possono essere efficacemente appresi da reti neurali generative profonde [7, 8].

L'interazione con questi modelli ha portato all'emergere di capacità cognitive e algoritmiche sofisticate, note come *capacità emergenti*. A differenza dei parser tradizionali, un LLM è in grado di:

1. **Catturare la grammatica e la struttura gerarchica:** comprendere la conformità sintattica e le dipendenze innestate dell'albero sintattico astratto (Abstract Syntax Tree, AST) pur elaborando il codice come una sequenza monodimensionale linearizzata di token discreti [9];
2. **Inferire l'intento algoritmico sottostante:** ricostruire lo scopo logico di procedure complesse mediante l'astrazione dal dettaglio implementativo, col-

mando il divario semantico (*semantic gap*) tra il formalismo computazionale di macchina e la modellazione concettuale dell'utente [10, 11];

3. **Eseguire compiti cross-modali e cross-linguaggio:** generare compiti di sintesi automatica di programmi a partire da specifiche in linguaggio naturale, implementare traduzioni funzionali tra linguaggi computazionali eterogenei e sintetizzare spiegazioni analitiche sul comportamento operativo del software [12, 13].

Ciononostante, il fondamento matematico-probabilistico di tali architetture cela limiti intrinseci e vulnerabilità strutturali che ne ostacolano l'adozione acritica in contesti industriali mission-critical:

- **Allucinazioni e fragilità su vincoli e precondizioni:** Trattandosi di stimatori statistici di densità di probabilità congiunta del tipo:

$$P(X) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}; \Theta) \quad (1.1)$$

(dove x_t è il token generato al passo t e Θ è l'insieme dei pesi della rete), i modelli generano output conformi ai pattern di massima verosimiglianza superficiale ma privi di verifica logica o contrattuale. Questo fenomeno induce frequenti *allucinazioni tecniche*, quali l'invenzione di costrutti inesistenti, la presupposizione errata di precondizioni, o la fabbricazione di parametri non presenti nella dichiarazione della funzione [14, 15].

- **Instabilità stocastica e sensibilità al prompting:** La natura probabilistica della decodifica (influenzata da parametri quali temperatura τ , top- p e strategie di campionamento) rende le risposte non perfettamente riproducibili, inducendo variazioni significative a fronte di modifiche minimali o non semantiche nel prompt di ingresso.
- **Cecità alle relazioni logico-relazionali fini:** Senza strumenti di analisi statica o verificatori formali esterni integrati, gli LLM faticano a tracciare vincoli di flusso dati, cammini d'esecuzione e relazioni di aliasing che richiederebbero ragionamenti formali e deduttivi [16].
- **Finestra di contesto finita e fenomeno “Lost in the Middle”:** Nonostante l'espansione nominale delle finestre di contesto delle recenti architetture Transformer, l'attenzione stocastica degrada sistematicamente in precisione nel recupero di informazioni posizionate nella porzione centrale della finestra di prompt (*Lost in the Middle*). Tale limitazione compromette la capacità di sintetizzare contesti architettureali complessi che si estendono su centinaia di file sorgente interconnessi.

1.2 Il Ruolo Critico della Documentazione del Software: Dalla Manutenzione al “Continuous Drift”

La documentazione del software non rappresenta un elemento meramente accessorio o decorativo del ciclo di vita dello sviluppo; essa è un pilastro portante dell'ingegneria del software su cui poggiano la manutenibilità, l'onboarding di nuovi ingegneri, la comprensione di sistemi legacy e l'affidabilità sistemica nel lungo periodo [17, 11]. In letteratura è ampiamente consolidato che gli ingegneri del software impiegano fino al 70% del proprio tempo in attività di comprensione del programma (*program comprehension*) piuttosto che nella scrittura attiva di nuove linee di codice [18, 9]. In questo scenario, una documentazione accurata e concisa agisce da interfaccia di astrazione ad alto livello, consentendo di comprendere i contratti operativi di una procedura senza doverne dissezionare le implementazioni interne.

Tuttavia, la prassi industriale è afflitta da una costante e problematica discrepanza tra il codice e le sue descrizioni: il *debito tecnico documentale*. La redazione manuale della documentazione tecnica costituisce un'attività complessa, tediosa e dispendiosa in termini temporali. Con il susseguirsi delle iterazioni di sviluppo, dei commit e dei refactoring architetturali, i team di ingegneria tendono fisiologicamente ad aggiornare le istruzioni eseguibili tralasciando l'adeguamento dei commenti associati. Questo fenomeno genera il cosiddetto **Documentation Drift**:

$$\Delta_{\text{drift}}(t) = \|\mathcal{S}_{\text{code}}(t) - \mathcal{S}_{\text{doc}}(t)\|_{\mathcal{H}} \quad (1.2)$$

dove $\mathcal{S}_{\text{code}}(t)$ e $\mathcal{S}_{\text{doc}}(t)$ rappresentano rispettivamente la semantica effettiva implementata nel codice e quella asserita nei blocchi documentali al tempo t , proiettate su uno spazio di astrazione contrattuale $\mathcal{H}$. All'aumentare di Δ_{drift} , la documentazione non solo perde la propria utilità descrittiva, ma si trasforma in un elemento attivo di disinformazione tecnica, inducendo i consumatori delle API ad assumere comportamenti errati che sfociano in bug silenti e regressioni disastrose [19, 14].

Questa criticità assume connotati particolarmente allarmanti nel contesto dei linguaggi di programmazione di livello di sistema, quali C e C++. A differenza di linguaggi a memoria gestita (come Java, C# o Python), C e C++ operano a stretto contatto con l'hardware e scaricano sul programmatore la gestione del ciclo di vita delle risorse:

- **Gestione manuale della memoria e proprietà (Ownership):** Il modello C/C++ non include un Garbage Collector deterministico a runtime. Il contratto di una funzione deve specificare chiaramente l'ownership dei puntatori passati e restituiti: se la funzione assume la proprietà di un buffer allocato, se alloca memoria dinamica mediante primitive (`malloc`, `calloc`, operatore `new`) che deve essere rilasciata dal chiamante (`free`, `delete`), o se trattiene puntatori pendenti [16].
- **Puntatori raw e vincoli di aliasing:** L'uso pervasivo di puntatori grezzi (*raw pointers*) e l'assenza di controlli sui limiti degli array (*bounds checking*)

impongono che la documentazione formalizzi rigorosamente le precondizioni d'ingresso (es. garanzia di puntatore non nullo, dimensione minima del buffer puntato, assenza di sovrapposizioni tra aree di memoria in conformità alla clausola `restrict`).

- **Trattamento degli errori ed effetti collaterali non tracciati dai tipi:** In assenza di un sistema uniforme di gestione delle eccezioni tipizzate (o in codice conforme a standard dove le eccezioni sono disabilitate per vincoli *hard real-time*), la segnalazione di anomalie è affidata a codici di errore interi restituiti, a variabili globali di stato (es. `errno`) o a parametri formali passati per riferimento come canali di uscita (*out parameters*). Se la documentazione omette il valore numerico o semantico del codice di errore restituito, il chiamante non è in grado di gestire in sicurezza i rami di fallimento.

In tali ambienti operativi critici, la documentazione non è una semplice narrazione parafrasata, bensì un vero e proprio **contratto formale d'interfaccia**. Automatizzarne la sintesi e la verifica in modo scientificamente rigoroso costituisce dunque un'esigenza ingegneristica non rinviabile.

1.3 Evoluzione Storica del Code Summarization: Le Quattro Epoche

Il problema della generazione automatica di riassunti e spiegazioni di codice sorgente (*Code Summarization*) vanta una storia pluridecennale nell'ingegneria del software. Come sintetizzato nella letteratura di riferimento [9, 11, 20], tale evoluzione può essere formalmente articolata in quattro epoche distinte, ciascuna caratterizzata da specifici assunti teorici, metodologie e vincoli di scalabilità.

1.3.1 L'Era delle Euristiche e dei Template (pre-2010)

Agli albori della disciplina, l'approccio predominante era strettamente deterministico, simbolico e guidato da regole sintattiche rigide. I sistemi di questa prima generazione si basavano sull'estrazione puramente strutturale di entità dal codice e sull'interpolazione di tali elementi all'interno di modelli testuali predeterminati (*templates*) [17].

Tra i contributi seminali di quest'epoca si annoverano:

- **SCHEMACODE di Robillard (1986):** L'impiego di pseudocodice schematico per rappresentare i costrutti di controllo del flusso dei programmi, associando commenti operativi derivati dall'applicazione di una grammatica formale rigida [21].
- **Slicing di programma e approcci basati su TASSAL:** Le tecniche di *program slicing* fondate sul lavoro pionieristico di Mark Weiser (1984) [22], in cui la scomposizione automatica del codice in base a flussi di controllo e dati permetteva di isolare porzioni minime di programma responsabili di un

determinato comportamento, agevolando l'estrazione di metriche di astrazione per la generazione di commenti.

- **Sistemi a template rule-based (Sridhara et al., 2010):** L'applicazione di tecniche di analisi dei blocchi di controllo per estrarre la firma del metodo, le assegnazioni principali e le istruzioni condizionali dominanti, mappandole su schemi prefissati del tipo: "This method executes <action> on <target> and returns <value>" [17].

Limite intrinseco: Questi sistemi soffrivano di un'estrema rigidità formale. Il testo prodotto risultava un mero *boilerplate* tautologico che ripeteva pedissequamente i nomi delle variabili e delle chiamate di funzione, privo di qualsiasi reale comprensione semantica o inferenza sull'intento operativo reale dell'autore.

1.3.2 L'Era dell'Information Retrieval (primi anni 2010)

Con l'avvento dei grandi repository open-source (la rivoluzione del "Big Code"), la comunità scientifica comprese che i programmatori tendono a risolvere problemi analoghi adottando soluzioni strutturalmente e lessicalmente simili. Nacque così il paradigma del Code Summarization guidato dal Recupero di Informazione (*Information Retrieval*, IR).

Il principio di base consiste nell'assumere una funzione di similarità lessicale o vettoriale tra snippet di codice:

$$c^* = \arg \max_{c \in \mathcal{C}_{\text{corpus}}} \text{Sim}(c_{\text{query}}, c) \quad (1.3)$$

dove c_{query} è il codice non documentato, $\mathcal{C}_{\text{corpus}}$ è una base di conoscenza di frammenti di codice dotati di commento umano, e il commento associato a c^* viene estratto e riadattato.

Sistemi quali BM25, Rencos e l'architettura a due stadi *EditSum* di Li et al. (2021) [23] rappresentano l'apice di questo filone: EditSum recupera un sommario prototipale dal corpus mediante somiglianza e successivamente applica una modifica controllata (*retrieve-and-edit*) per specializzare i token specifici del nuovo frammento.

Tuttavia, lo studio empirico estensivo pubblicato da Zhu et al. (TOSEM 2024) [20] ha tracciato il confine computazionale invalicabile di questa classe di modelli:

- Gli approcci IR dimostrano prestazioni eccellenti e talvolta superiori ai modelli neurali solo in presenza di un'elevata sovrapposizione lessicale e strutturale tra la query e il database (Lexical Overlap > 60%).
- A fronte di algoritmi inediti, logiche di dominio personalizzate o strutture sintattiche atipiche, il modello di IR fallisce sistematicamente, recuperando sommari completamente fuorvianti e semanticamente scorrelati.

1.3.3 L'Era del Deep Learning Iniziale (metà anni 2010)

Verso la metà degli anni 2010, il successo della traduzione automatica neurale (Neural Machine Translation, NMT) nel processamento del linguaggio naturale

ispirò una ridefinizione formale del compito: la generazione di documentazione fu inquadrata come una traduzione da un linguaggio formale sorgente a un linguaggio naturale di destinazione.

Questo periodo ha visto l'introduzione di architetture neurali encoder-decoder sequenziali e grafiche:

1. **CODE-NN di Iyer et al. (2016)** [24]: Il primo modello basato su reti ricorrenti (RNN) con meccanismo di attenzione (attention-based sequence-to-sequence) per generare sommari sintetici da sequenze di codice sorgente atomiche.
2. **DeepCom di Hu et al. (2018)**: L'incorporazione della struttura sintattica mediante l'attraversamento linearizzato dell'albero AST (*Structure-Based Traversal*, SBT), trasformando l'albero sintattico in una sequenza preservante la gerarchia da processare tramite LSTM [9].
3. **code2seq di Alon et al. (2019)** [25]: Superamento della linearizzazione sequenziale mediante la scomposizione del codice in un insieme di cammini compositivi su AST. La rete codifica i cammini tra nodi foglia tramite LSTM bidirezionali e impiega un meccanismo di attenzione per selezionare dinamicamente i percorsi più informativi durante la decodifica.
4. **Transformer e Modelli Pre-addestrati**: L'avvento di modelli quali Transformer dedicati al codice [26], CodeBERT [7] e GraphCodeBERT [8], che introducono compiti di mascheramento bilingue e integrano flussi di dati (*data flow*) per catturare le relazioni di assegnazione tra variabili.

1.3.4 Il “Soffitto Tecnologico” Pre-LLM (The Deep Learning Ceiling)

Nonostante gli indiscutibili progressi rispetto alle euristiche simboliche, l'era del Deep Learning classico si è scontrata con un insormontabile “soffitto tecnologico”, originato da tre colli di bottiglia architetturali:

1. **Vincoli di Contesto (Context Constraints)**: Le architetture RNN, LSTM e le prime implementazioni Transformer soffrivano di una marcata incapacità di gestire dipendenze temporali lunghe. Con l'aumentare della lunghezza del file sorgente, il gradiente dell'attenzione subiva un rapido decadimento matematico, limitando l'analisi a finestre di poche centinaia di token.
2. **Il Problema dei Token Rari e OOV (The Rare Token Problem)**: I linguaggi di programmazione presentano una cardinalità lessicale potenzialmente infinita dovuta a identificatori composti definiti dagli sviluppatori, macro di registro, convenzioni snake_case e camelCase. I modelli con vocabolario fisso ($|V| \leq 30.000 - 50.000$) collassavano frequentemente su token sconosciuti (<UNK>), allucinando identificatori cruciali per comprendere il dominio applicativo.

3. **Visione Miope e Atomica (Myopic Scope):** Il vincolo più penalizzante era l'isolamento metodologico. La quasi totalità della letteratura considerava esclusivamente il livello del singolo metodo atomico (*method-level summarization*) [11]. In un progetto C/C++ reale, il corpo di una funzione è incomprensibile se disgiunto dalle definizioni di tipo (typedef, struct, enum) collocate nei file header di inclusione (.h), dalla gerarchia di ereditarietà o dalle funzioni chiamate all'interno del modulo [27, 16]. Questa miopia ha impedito per anni l'impiego reale di tali modelli in codebase industriali complesse.

1.4 Dall'Analisi Atomica alla Scala Repository: Micro-Scale vs Macro-Scale

La comprensione ingegneristica del software richiede una netta differenziazione tra due regimi dimensionali complementari: il regime a scala microscopica (*Micro-Scale*), incentrato sull'accuratezza della singola unità procedurale, e il regime a scala macroscopica (*Macro-Scale*), orientato alla preservazione delle interconnessioni architetturali globali.

1.4.1 Micro-Scale: Dalla Funzione alla Documentazione di Dettaglio

A livello microscopico, l'obiettivo è la generazione della documentazione di interfaccia per una singola unità computazionale isolata. L'evoluzione dei modelli di frontiera offre oggi tre metodologie primarie di adattamento:

1. **Instruction Prompting (Zero/Few-Shot):** Consiste nell'alimentare un LLM pre-addestrato (quali StarCoder [3], DeepSeek-Coder, Claude, GPT-4) mediante istruzioni in linguaggio naturale corredate o meno da esempi dimostrativi nel contesto (*In-Context Learning*) [10]. Sebbene non richieda costi computazionali di addestramento e risulti di immediata applicabilità, è altamente suscettibile a instabilità di formato e non garantisce la conformità sintattica ai vincoli formali del compilatore.
2. **Task-Oriented Full Fine-Tuning:** Comporta l'aggiornamento globale di tutti i parametri Θ del modello su un dataset supervisionato di coppie <codice, documentazione>. Pur massimizzando la convergenza statistica sullo stile del corpus di addestramento, questa metodologia comporta costi computazionali insostenibili per modelli di grande taglia e incorre nel grave rischio del *catastrophic forgetting*, degradando la capacità di ragionamento logico generale del modello pre-addestrato.
3. **Continuous Prompting e Parameter-Efficient Fine-Tuning (PEFT):** Per conciliare efficienza e specializzazione, approcci all'avanguardia hanno proposto la parametrizzazione di vettori continui di prompt. Un esempio emblematico è rappresentato dal framework **PromptCS** proposto da Xu

et al. (2024) [3]. In PromptCS, i parametri del modello LLM di base rimangono completamente congelati (*frozen*); un agente neurale dedicato (denominato *Prompt Agent*, strutturato su reti BiLSTM) riceve il codice sorgente e sintetizza dinamicamente un vettore continuo di embedding:

$$\mathbf{P}_{\text{soft}} = \text{BiLSTM}(X_{\text{code}}; \Phi) \quad (1.4)$$

che viene prefissato alla rappresentazione del codice, guidando lo spazio di generazione del modello verso descrizioni informative senza alterarne i pesi originali.

Nonostante l’eleganza teorica del continuous prompting, esso presenta vincoli operativi severi: è strettamente vincolato all’accesso diretto ai vettori di embedding del modello, risultando quindi inapplicabile alle potenti API dei modelli commerciali closed-source. Inoltre, la capacità espressiva complessiva resta limitata dalla conoscenza statica memorizzata dal modello congelato.

1.4.2 Macro-Scale: Dalla Codebase alla Mappatura Architetture

Quando la complessità del sistema software scala all’intero repository, il paradigma del *single-pass prompting* (inviare l’intero codice in un’unica invocazione) collassa a causa dell’esaurimento della finestra di contesto, del rumore informativo e del costo esponenziale dell’auto-attenzione. Per generare una documentazione di livello aziendale è indispensabile un’architettura macroscopica basata su tre prerequisiti strutturali:

1. **Analisi Statica e AST Parsing Deterministico:** L’adozione di un motore di compilazione reale (quale `Clang/LLVM`) per dissezionare deterministicamente le unità di traduzione C/C++, estraendo la totalità dei simboli, le firme formali, le visibilità e i mapping di inclusione (`#include`) privi di allucinazioni [16].
2. **Dependency & Call Graphs Inter-Procedurali:** La costruzione esplicita di grafi di chiamata globali $G = (V, E)$, in cui ogni vertice rappresenta una funzione o entità architetture e ogni arco orientato modella una relazione di invocazione o consumo di tipo. Sistemi pionieristici come *PROCONSUL* (Lomshakov et al., EMNLP 2024) [16] hanno certificato che arricchire il prompt dell’LLM con il contesto semantico estratto dai callee e dai caller (informazioni su tipi composti, contratti di ritorno e macro) riduce le allucinazioni tecniche di oltre il 40% rispetto ai prompt atomici decontestualizzati.
3. **Orchestrazione Agentica Modulare:** La scomposizione del processo documentale in una sequenza coordinata di agenti specializzati, in cui ruoli differenziati gestiscono l’estrazione, la sintesi, la verifica e l’ordinamento topologico di dipendenza per garantire la propagazione coerente delle informazioni dal basso verso l’alto [14, 28].

1.5 Stato dell’Arte dei Framework di Repository: CodeWiki, RepoAgent e DocAgent

La ricerca contemporanea più avanzata nell’intersezione tra ingegneria del software e sistemi multi-agente basati su LLM ha prodotto paradigmi innovativi orientati alla documentazione olistica di interi repository. Di seguito si esaminano i quattro modelli di riferimento dello stato dell’arte che ispirano e contestualizzano l’approccio proposto nella presente tesi.

1.5.1 CodeWiki (Hoang et al., 2026)

CodeWiki [28] affronta la sfida di generare una documentazione tecnica olistica e consapevole dell’architettura per codebase su larga scala distribuite su molteplici linguaggi. Il contributo cardine di CodeWiki risiede nell’introduzione di una *scomposizione gerarchica ricorsiva con delega dinamica dei task*:

- Un agente di vertice (*Repository Manager*) pianifica e suddivide l’albero del progetto in moduli logici;
- Una rete di agenti intermedi (*Module Agents*) coordina l’analisi dei sottosistemi;
- Gli agenti esecutori terminali (*Leaf Agents*) estraggono e documentano le singole unità implementative.

La sintesi opera con una logica bottom-up: i sommari sintetizzati dai Leaf Agents vengono ricorsivamente aggregati per distillare la panoramica concettuale del modulo e del repository. Inoltre, CodeWiki innova integrando una componente di *sintesi multimodale*, producendo parallelamente alla prosa testuale diagrammi visuali in notazione Mermaid e grafi del flusso dati architetturali. Per validare il sistema, gli autori hanno introdotto il benchmark *CodeWikiBench*, fondato su metriche multidimensionali valutate tramite modelli LLM proprietari.

1.5.2 RepoAgent (Luo et al., 2024)

RepoAgent [19] affronta il problema cronico del deterioramento documentale introducendo il paradigma della *Continuous Documentation* integrata nei cicli di Continuous Integration e Continuous Deployment (CI/CD) e nel version control Git.

A differenza degli approcci statici offline, RepoAgent include un modulo euristico e algoritmico denominato *ChangeDetector*:

$$\Delta_{\text{commit}} \longrightarrow \mathcal{T}_{\text{impact}} = \text{TraverseCallGraph}(\Delta_{\text{AST}}) \quad (1.5)$$

Quando uno sviluppatore effettua un commit, RepoAgent calcola il diff sintattico sull’AST, identifica le funzioni direttamente alterate e calcola la chiusura transitiva dei loro dipendenti nel grafo di chiamata inter-procedurale. In questo modo, l’orchestrazione multi-agente non rigenera l’intera documentazione del progetto, bensì attiva una propagazione differenziale minima e reattiva, garantendo l’allineamento perenne tra codice e specifiche senza incorrere in costi computazionali proibitivi.

1.5.3 DocAgent (Yang et al., 2025)

Presentato alla conferenza ACL 2025, **DocAgent** [14] struttura l’elaborazione del software secondo un’architettura multi-agente collaborativa basata sull’*ordinamento topologico di dipendenza* del codice sorgente. Il grafo delle chiamate viene linearizzato in modo che nessuna funzione venga documentata prima che i suoi callee non siano stati precedentemente analizzati e documentati, consentendo la costruzione incrementale e contestualizzata del prompt.

DocAgent formalizza inoltre una specifica triade qualitativa di requisiti per la validazione della documentazione, strutturata su tre assi ortogonali:

1. **Completeness (Completezza)**: Il grado di copertura di tutti i componenti essenziali della funzione (argomenti, tipi, valori di ritorno, eccezioni/errori e logica essenziale);
2. **Helpfulness (Utilità operativa)**: La chiarezza delle indicazioni fornite, l’assenza di ridondanza boilerplate e l’effettiva capacità di guidare un programmatore terzo nell’integrazione della routine;
3. **Truthfulness (Veridicità e Fedeltà)**: La totale assenza di allucinazioni fattuali, precondizioni inventate o contraddizioni rispetto all’effettivo comportamento deterministico del codice sorgente.

La validazione di tale triade è demandata a un agente *Verifier* dedicato che opera secondo il paradigma LLM-as-a-Judge con protocolli di auditing strutturati [29, 30, 31].

1.5.4 RepoSummary (Zhu et al., 2025)

RepoSummary [32] affronta una delle problematiche più complesse nell’analisi dei sistemi legacy: la discrepanza tra la struttura fisica delle directory su disco e l’architettura logico-funzionale effettiva del sistema. Spesso, nei monoliti software storici, file collocati in sottocartelle distanti collaborano strettamente all’erogazione della medesima funzionalità ad alto livello.

RepoSummary supera l’ordinamento topografico rigido introducendo una tecnica di *clusterizzazione feature-oriented*:

$$\mathbf{A}_{\text{ibrida}} = \alpha \mathbf{A}_{\text{semantica}} + (1 - \alpha) \mathbf{A}_{\text{strutturale}} \quad (1.6)$$

in cui la matrice di adiacenza del grafo di dipendenza strutturale viene fusa con la matrice di similarità semantica degli embedding del codice. Attraverso algoritmi di partizionamento su grafi, il repository viene segmentato in “Epiche Funzionali” (Feature Clusters). La documentazione genera mappa esplicitamente le catene di tracciabilità (*traceability links*) che connettono le feature astratte ai metodi atomici distribuiti, fornendo una guida fondamentale sia per la comprensione che per il refactoring guidato del software [4, 15].

1.5.5 Quadro Comparativo Sinottico

Nella Tabella 1.1 viene proposta una disamina sinottica comparativa tra i principali framework dello stato dell'arte e l'approccio ingegneristico introdotto nella presente tesi.

Come si evince dall'analisi tassonomica, i sistemi correnti si collocano o sul versante micro-analitico privo di contesto sistemico (es. PromptCS), oppure su quello macroscopico che delega la totalità della validazione a valutatori LLM stocastici privi di un ancoraggio deterministico alle proprietà statiche del compilatore (es. DocAgent, CodeWiki). La soluzione qui formalizzata si colloca all'intersezione di tali paradigmi, integrando il rigore deterministico del compilatore C/C++ con l'orchestrazione cognitiva multi-agente.

1.6 Obiettivo della Tesi e “Strategic Blueprint” Proposto

1.6.1 Definizione dell'Obiettivo Primario

La presente tesi magistrale si prefigge l'obiettivo di superare lo stato dell'arte attraverso la progettazione, formalizzazione e implementazione empirica di un **framework olistico end-to-end multi-agente**, specificamente architettato per la generazione automatica, la validazione formale e il mantenimento continuo della documentazione tecnica di codebase complesse scritte in C e C++.

Dato in ingresso un repository software reale $\mathcal{R}_{C/C++}$ caratterizzato da una complessa topologia di file di intestazione (`.h/.hpp`), unità di traduzione (`.c/.cpp`), gerarchie di inclusione e puntatori opachi, il framework produce in uscita un corpus documentale tecnico (conforme allo standard Doxygen ed esteso a sintesi architetturali globali) *certificato, gerarchico, formalmente verificato e consapevole dell'architettura*.

1.6.2 L'Architettura Modulare a Quattro Livelli (Strategic Blueprint)

Per garantire il rigore scientifico e la separazione dei compiti architetturali, il framework sviluppato è strutturato su quattro livelli cooperativi:

1. **Parser & Static Analysis Layer (Fase Deterministica):** Questo modulo si interfaccia con il compilatore `Clang/LibTooling`. Invece di delegare all'LLM l'interpretazione testuale grezza del codice sorgente, il Parser Layer esegue il parsing completo delle unità di traduzione, generando:
 - L'AST formale di ciascuna funzione, con le firme esatte dei tipi, i qualificatori di puntatore (`const`, `*`, `&`), i contratti di visibilità e i mapping di ritorno;
 - Il *Control Flow Graph* (CFG) per intercettare i blocchi di guardia condizionali e le uscite con errori (`return -1`, codici di stato);

Tabella 1.1: Quadro comparativo sinottico tra i framework avanzati di repository summarization dello stato dell’arte e la soluzione proposta.

Framework		Granularità	Analisi Contesto	Meccanismo Validazione	Peculiarità Distinti- va
PromptCS	[3]	Micro (Method- level)	Nessuno (Iso- lamento ato- mico)	Metriche NLP superficiali (BLEU, ROU- GE)	Prompt Agent neurale (BiLSTM) per con- tinuous prompting con LLM frozen.
RepoAgent	[19]	Macro (File / Repo)	Call Graph statico mono- direzionale	Nessuna verifi- ca formale au- tomatica	Paradigma <i>Conti- nuous Documen- tation</i> e rileva- mento differen- ziale dei commit via Git.
DocAgent	[14]	Macro (Modulo Repo)	/ Ordinamento topologico chiamate	Agente Verifier via LLM-as-a- Judge	Triade quali- tativa (Comple- teness, Help- fulness, Truth- fulness) per repo complessi.
CodeWiki	[28]	Macro gerar- chico (Siste- ma intero)	Scomposizione ricorsiva ge- rarchica	Valutazione LLM (bench- mark CodeWi- kiBench)	Decomposizione modulare bottom- up e sintesi multimo- dale (testo e diagram- mi).
ApproccioProposto		Ibrido Inte- grato (Micro & Macro)	AST Clang + Call/Type Graphs + Header Ex- pansion	Certificazione 4-Layer: Ve- rifier AST, Verificatore formale Clang e Round-Trip Testing	Ciclo chiuso determi- nistico: verifica contrat- tuale estesa su

- Il *Call & Type Graph* globale inter-procedurale per correlare costrutti e tipi composti definiti negli header correlati.
2. **Micro-Scale Agent Layer (Sintesi Atomica e Focalizzata):** In questo strato, agenti specializzati ricevono il corpo della singola funzione arricchito dal suo contesto compilativo e strutturale circostante (*Expanded Signature and Callee Context*) [16]. L'agente genera la documentazione tecnica micro-level garantendo che ogni parametro riceva un'esplicitazione della propria direzionalità ([in], [out], [in,out]), dell'ownership di memoria e dei requisiti di pre/postcondizione.
 3. **Macro-Scale Multi-Agent Orchestration Layer (Sintesi Gerarchica):** Adottando un'architettura gerarchica bottom-up ispirata ai principi di CodeWiki [28] e DocAgent [14], una comunità di agenti conversazionali (Repository Manager, Module Synthesizers e Interface Documenters) sintetizza le correlazioni tra i file del sistema. La propagazione delle sintesi ascende dai moduli elementari alle cartelle di sottosistema fino a compilare il sommario globale dell'architettura del software, includendo la generazione automatica di diagrammi di flusso e grafi dei componenti espressi in standard Mermaid.
 4. **Evaluation & Verification Loop (Validazione Multidimensionale):** A coronamento della pipeline, la documentazione generata non viene considerata accettata fino al superamento di un loop formale e comportamentale:
 - *AST Verifier*: Controllo deterministico di coerenza biettiva tra parametri AST e clausole Doxygen;
 - *Static Contract Guard*: Analisi della copertura della gestione degli errori e dei vincoli di allocazione;
 - *LLM-as-a-Judge Protocol*: Audit a rubric chiusa su Completezza, Utilità e Veridicità;
 - *Differential Round-Trip Testing*: Esecuzione a scatola nera in cui agenti programmatori autonomi rigenerano implementazioni a partire dalla documentazione prodotta e le sottopongono a suite di test formali (Hypothesis/Pytest) per validarne la consistenza semantica effettiva.

1.6.3 I Tre Principi Cardine della Tesi

L'intera dissertazione e lo sviluppo tecnologico associato si sviluppano su tre postulati scientifici fondamentali:

1. **Principio I: Il Contesto Determina la Qualità (Context-Driven Quality):** La documentazione generata da un modello autoregressivo per una funzione C/C++ non può eccellere se limitata all'elaborazione del solo snippet locale. La fornitura sistematica del contesto strutturale (tipi negli header, firme dei callee, vincoli dell'AST) costituisce la preconditione fisica indispensabile per abbattere le allucinazioni e produrre specifiche affidabili.

2. **Principio II: L’Orchestrazione Gerarchica Supera la Finestra di Contesto (Hierarchical Decomposition):** La scalabilità a codebase di centinaia di migliaia di linee di codice non si persegue incrementando all’infinito la finestra di token dell’LLM (operazione vulnerabile a fenomeni di distrazione e decadimento attentivo), bensì ingegnerizzando architetture agentiche modulari e ricorsive che scompongono il problema secondo le gerarchie naturali del software (Funzione → File → Modulo → Sottosistema → Repository).
3. **Principio III: La Documentazione è un’Entità Viva e Verificabile (Living & Executable Specification):** La documentazione tecnica non deve costituire un artefatto statico, redatto una volta e abbandonato all’invecchiamento. Essa deve vivere all’interno del flusso continuo del repository (Continuous Documentation), integrarsi con il compilatore statico ed essere sottoposta a continua validazione empirica, trasformandosi in una specifica azionabile e testabile a livello computazionale.

1.6.4 Struttura e Organizzazione della Tesi

La presente tesi è strutturata nei seguenti capitoli tematici:

- **Capitolo 2 – Fondamenti Teorici e Stato dell’Arte:** Analizza approfonditamente l’evoluzione delle architetture neurali generative, i modelli Transformer per il codice sorgente (CodeBERT, GraphCodeBERT, StarCoder, DeepSeek), i principi dell’analisi statica dei programmi (AST, CFG, DFG) basata sul framework Clang/LLVM e le basi concettuali delle architetture ad agenti autonomi collaborativi (AutoGen, SWE-agent).
- **Capitolo 3 – Architettura del Framework Multi-Agente:** Illustra dettagliatamente la progettazione ingegneristica e l’implementazione del sistema proposto. Vengono descritti l’interfacciamento deterministico con l’AST parser, la gestione del grafo di chiamata inter-procedurale, i prompt template specializzati e i meccanismi di coordinamento asincrono tra gli agenti (Reader, Architect, Synthesizer, Verifier).
- **Capitolo 4 – Tassonomia e Formulazione Analitica delle Metriche di Valutazione:** Presenta una formalizzazione analitica e matematica rigorosa delle metriche di valutazione articolate su quattro livelli: metriche formali contrattuali AST, metriche software di actionability, metriche semantiche dense (Sentence-BERT, BERTScore, CodeBERTScore, BLEURT, LLM-as-a-Judge) e metriche empiriche comportamentali di Round-Trip Testing ed esecuzione differenziale.
- **Capitolo 5 – Validazione Sperimentale, Analisi Comparativa e Risultati:** Riporta i risultati quantitativi e qualitativi di una vasta campagna di sperimentazione condotta su codebase reali C/C++. Vengono valutate comparativamente le prestazioni del framework rispetto ai modelli baseline e ai framework dello stato dell’arte (PromptCS, RepoAgent, DocAgent, CodeWiki),

supportate da studi di ablazione, analisi statistica di correlazione con il giudizio umano e visualizzazione grafica dei trade-off prestazionali.

- **Capitolo 6 – Conclusioni e Sviluppi Futuri:** Riassume i contributi scientifici della tesi, discute le implicazioni industriali del framework proposto ed evidenzia le prospettive future di ricerca, tra cui l'integrazione di verificatori formali a livello di modello (Model Checking) e la sintesi completamente autonoma di specifiche di sicurezza formale (*Formal Proofs*).

Capitolo 2

Tassonomia e Formulazione Analitica delle Metriche di Valutazione

2.1 Introduzione e Motivazione

La generazione automatica della documentazione del codice sorgente rappresenta una delle sfide più rilevanti e complesse nell’ambito dell’ingegneria del software moderna e dell’elaborazione del linguaggio naturale (NLP) [9, 11, 33]. Nei linguaggi di programmazione di livello di sistema come C e C++, la documentazione tecnica (tipicamente standardizzata nel formato Doxygen) non assolve unicamente a una funzione esplicativa o didattica ad alto livello; essa costituisce un vero e proprio *contratto formale d’interfaccia* tra il callee e i suoi chiamanti (caller). In tali contesti, discrepanze apparentemente trascurabili nella formulazione testuale – quali l’ambiguità sulla deallocazione della memoria dinamica, l’omissione di un vincolo di sincronizzazione o la mancata dichiarazione della gestione di puntatori nulli – possono propagarsi in vulnerabilità di sicurezza catastrofiche, memory leak, dangling pointer e violazioni di accesso a runtime.

Tradizionalmente, la valutazione della documentazione sintetizzata da modelli computazionali o da Large Language Models (LLM) è stata demandata a due macro-approcci:

1. **Valutazione Umana (Human Evaluation):** condotta mediante peer review o rubric-based scoring da parte di sviluppatori senior. Sebbene considerata storicamente il punto di riferimento (*ground truth*), essa risulta proibitiva in termini di costi scalari, intrinsecamente non riproducibile, soggetta a forte variabilità inter-annotatore e priva della capacità di coprire esaustivamente code-base industriali di decine di migliaia di linee di codice (LOC).
2. **Metriche NLP Tradizionali basate su n-grammi superficiali:** l’impiego diretto di metriche mutate dalla traduzione automatica (Machine Translation) come BLEU [34] o dal riassunto astrattivo come ROUGE [35].

Tali metriche si limitano a quantificare la sovrapposizione lessicale esatta (*exact string matching*) tra il testo generato e un riferimento scritto dall'autore originario.

Questo paradigma classico rivela limiti strutturali insormontabili quando applicato alla documentazione del software C/C++. Il dominio è infatti caratterizzato da una complessa **dualità semantica**: da un lato la flessibilità espressiva del linguaggio naturale umano (dove concetti identici possono essere parafrasati attraverso una molteplicità di formulazioni sintattiche), dall'altro la rigidità deterministica della semantica software e dei vincoli di macchina. Una metrica meramente lessicale penalizza severamente una parafrasi tecnica corretta, mentre può assegnare punteggi ingannevolmente elevati a una documentazione che inverte in modo distruttivo un vincolo logico (ad esempio sostituendo `<` con `>` o `must free` con `must not free`).

Per superare tali limitazioni, la presente tesi introduce un framework di valutazione multi-dimensionale, formalizzato in una tassonomia a quattro livelli gerarchici. Tale architettura unisce l'analisi statica dei tipi (AST), la quantificazione della completezza operativa del software, la similarità contestuale neurale e la validazione empirica tramite esecuzione differenziale e sintesi round-trip.

2.2 Stato dell'Arte e Tassonomia a Quattro Livelli

La modellazione della qualità della documentazione software richiede un disaccoppiamento esplicito delle dimensioni di analisi. Come illustrato nella classificazione adottata, definiamo quattro livelli di astrazione tra loro complementari:

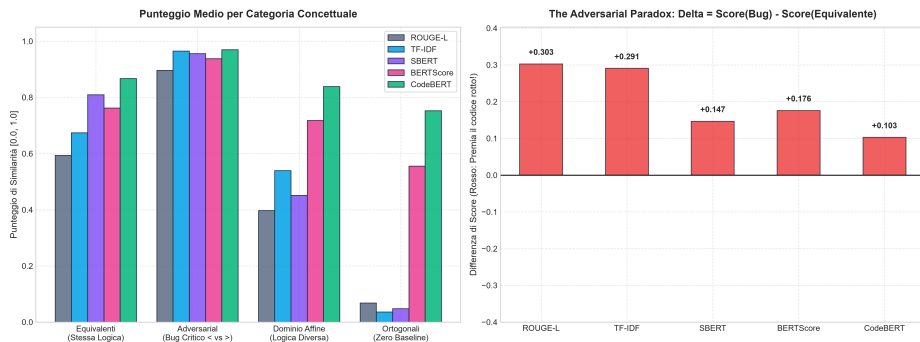


Figura 2.1: Spettro di sensibilità e comportamento differenziale delle metriche attraverso diverse condizioni sperimentali e regimi semantici.

1. **Livello I: Correttezza Formale e Contratti AST (Clang Static Analysis):** Questo livello opera direttamente sulle strutture dati del compilatore (Abstract Syntax Tree estratto mediante Clang/LibTooling). L'obiettivo primario è certificare che ogni entità sintattica dichiarata nella firma della funzione (parametri formali, qualificatori di tipo, costanza, direzionalità dei

puntatori e tipo di ritorno) trovi un corrispettivo esatto e coerente nei tag strutturati Doxygen, intercettando rigorosamente allucinazioni di parametri inesistenti o omissioni formali.

2. **Livello II: Qualità Software e Actionability:** Valuta l'utilità operativa e la completezza della specifica per lo sviluppatore chiamante. Include la quantificazione della presenza di guardie sui casi limite (*Edge Cases*), la documentazione sistematica delle condizioni di fallimento e dei codici di errore restituiti (*Error Documentation Rate*), la completezza dell'allocazione e rilascio di risorse (*Concept Checklist*) e l'usabilità complessiva dell'interfaccia (*Actionability Score*).
3. **Livello III: Semantica Distribuita, Embedding Densi e LLM-as-a-Judge:** Misura l'aderenza concettuale superando la barriera del lessico superficiale. Impiega modelli Transformer pre-addestrati e fine-tunati sia sul testo generale (Sentence-BERT, BERTScore, BLEURT) sia sul codice sorgente bilingue (CodeBERTScore). Questo livello include inoltre una formalizzazione rigorosa del paradigma *LLM-as-a-Judge*, che interroga modelli di frontiera con rubric a cinque livelli operando campionamenti stocastici Monte Carlo per separare fedeltà tecnica (*Faithfulness*) e allineamento all'intento dell'autore (*Alignment*).
4. **Livello IV: Validazione Downstream Comportamentale (Differential Round-Trip Testing):** Costituisce il livello supremo di verifica empirica. La documentazione generata viene considerata una specifica formale a scatola nera (black-box specification): un agente LLM generatore (*Coder*) deve ricostruire il codice sorgente guidato esclusivamente dalla documentazione prodotta (senza aver mai visionato l'implementazione originaria), mentre un agente *Tester* sintetizza suite di proprietà e casi di test unitari (Pytest/Hypothesis). Il grado di preservazione della semantica viene misurato calcolando la percentuale di test superati (*Round-Trip Pass Rate*) e l'accordo differenziale di esecuzione rispetto alla logica di riferimento (*Dual Agreement Rate*).

2.3 Descrizione Analitica delle Metriche di Benchmark

Nel seguito viene fornita la formalizzazione matematica rigorosa e la descrizione algoritmica di ciascuna metrica implementata nel framework di benchmark. Per garantire massima leggibilità, uniformità strutturale e precisione contrattuale, ogni metrica è articolata secondo uno schema analitico uniforme comprendente: obiettivo concettuale, tipologia e formato degli input, formulazione matematica dell'algoritmo di calcolo, intervallo di variazione ed interpretazione ingegneristica.

2.3.1 Livello I: Metriche Formali e Contratti AST

Verifier Pass Rate

- **Obiettivo e Significato:** Misura la percentuale di funzioni la cui documentazione generata supera integralmente e senza alcuna violazione il set di regole sintattiche e semantiche imposte dal verificatore formale basato su Abstract Syntax Tree (AST) estratto da Clang. Rappresenta la condizione necessaria (ma non sufficiente) affinché la documentazione tecnica possa essere integrata in una pipeline di continuous integration (CI/CD) senza degradare la qualità formale dei metadati del repository.
- **Input e Tipologia di Coppia:** [AST vs Doc] L'input è costituito dall'oggetto AST estratto da Clang (contenente la lista ordinata dei parametri formali $P_{AST} = \{p_1, \dots, p_n\}$ e il tipo di ritorno statico T_{ret}) e dal blocco di commento Doxygen parsato nei suoi tag strutturati (@param, @return, @brief).
- **Formulazione Matematica ed Elaborazione:** Dato un corpus di N funzioni, sia $V(f_i) \in \{0, 1\}$ una funzione indicatrice booleana definita come:

$$V(f_i) = \begin{cases} 1 & \text{se } |\mathcal{E}(f_i)| = 0 \\ 0 & \text{se } |\mathcal{E}(f_i)| > 0 \end{cases} \quad (2.1)$$

dove $\mathcal{E}(f_i)$ rappresenta l'insieme degli errori diagnostici emessi dal parser/verifier formale (mancanza di tag obbligatori, disallineamento nei nomi dei parametri, clausole @return incoerenti con funzioni void o non-void). Il *Verifier Pass Rate* aggregato sull'intero corpus è calcolato come:

$$\text{Verifier Pass Rate} = \frac{1}{N} \sum_{i=1}^N V(f_i) \times 100\% \quad (2.2)$$

- **Range di Output e Interpretazione:** [0.0%, 100.0%]. Un valore del 100% indica che tutte le docstring sintetizzate rispettano perfettamente la conformità formale e sintattica dell'AST, azzerando le allucinazioni strutturali e le discrepanze di segnatura.

Parameter Precision, Recall ed F1-Score vs AST

- **Obiettivo e Significato:** Valutano l'accuratezza di slot-filling dei parametri documentati rispetto all'effettiva interfaccia formale estratta dall'albero di sintassi astratta, penalizzando sia l'omissione di parametri reali (falsi negativi) sia l'invenzione di argomenti fittizi (falsi positivi).
- **Input e Tipologia di Coppia:** [AST vs Doc] Insieme dei parametri formali estratti dall'AST $P_{AST} = \{p_1, \dots, p_n\}$ e insieme dei parametri dichiarati nei tag @param della documentazione generata $P_{DOC} = \{d_1, \dots, d_m\}$.
- **Formulazione Matematica ed Elaborazione:** Definiti i veri positivi come $TP = |P_{AST} \cap P_{DOC}|$, i falsi positivi come $FP = |P_{DOC} \setminus P_{AST}|$ e i falsi negativi

come $FN = |P_{AST} \setminus P_{DOC}|$, la Precisione, il Richiamo e l' F_1 -score sono formulati come:

$$\text{Precision}_{param} = \begin{cases} \frac{|P_{AST} \cap P_{DOC}|}{|P_{DOC}|} & \text{se } |P_{DOC}| > 0 \\ 1.0 & \text{se } |P_{AST}| = |P_{DOC}| = 0 \\ 0.0 & \text{se } |P_{DOC}| = 0 \wedge |P_{AST}| > 0 \end{cases} \quad (2.3)$$

$$\text{Recall}_{param} = \begin{cases} \frac{|P_{AST} \cap P_{DOC}|}{|P_{AST}|} & \text{se } |P_{AST}| > 0 \\ 1.0 & \text{se } |P_{AST}| = |P_{DOC}| = 0 \\ 0.0 & \text{se } |P_{AST}| = 0 \wedge |P_{DOC}| > 0 \end{cases} \quad (2.4)$$

$$F_{1,param} = \begin{cases} \frac{2 \cdot \text{Precision}_{param} \cdot \text{Recall}_{param}}{\text{Precision}_{param} + \text{Recall}_{param}} & \text{se } (\text{Precision}_{param} + \text{Recall}_{param}) > 0 \\ 0.0 & \text{altrimenti} \end{cases} \quad (2.5)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Un punteggio $F_{1,param} = 1.0$ sancisce una corrispondenza biunivoca perfetta tra i parametri fisici della funzione C/C++ e la loro dichiarazione nel blocco Doxygen.

Return Contract Match

- **Obiettivo e Significato:** Verifica la correttezza del contratto sul valore di ritorno: garantisce che una funzione non-void possieda una clausola `@return` esaustiva e, specularmente, che funzioni con tipo di ritorno `void` (oppure costruttori/distruttori C++) non documentino ingannevolmente valori restituiti inesistenti.
- **Input e Tipologia di Coppia:** **[AST vs Doc]** Tipo di ritorno statico T_{ret} ricavato da Clang e lista delle clausole documentate $\{r_1, \dots, r_k\} \subseteq \text{Doc}$.
- **Formulazione Matematica ed Elaborazione:** Sia $\text{isVoid}(T_{ret})$ un predicato che vale vero se il tipo di ritorno è `void` (escludendo esplicitamente i puntatori generici `void*`). Il valore di conformità $M_{ret} \in \{0.0, 1.0\}$ è formulato come:

$$M_{ret} = \begin{cases} 1.0 & \text{se } \text{isVoid}(T_{ret}) \wedge k = 0 \\ 1.0 & \text{se } \neg \text{isVoid}(T_{ret}) \wedge k > 0 \wedge \text{len}(r_1) \geq 5 \\ 0.0 & \text{altrimenti} \end{cases} \quad (2.6)$$

- **Range di Output e Interpretazione:** Range discreto $\{0.0, 1.0\}$. Un punteggio unitario attesta la perfetta coerenza logica e semantica tra la firma di ritorno e la presenza o assenza della clausola Doxygen associata.

Hallucination Rate

- **Obiettivo e Significato:** Quantifica la frequenza relativa di allucinazione strutturale, misurando la proporzione di parametri fittizi, identificatori inventati o simboli non esistenti introdotti nel testo generato rispetto alla totalità degli elementi documentati.

- **Input e Tipologia di Coppia:** [Code/AST vs Doc] Insieme degli errori intercettati dal verificatore $\mathcal{E}_{halluc} \subseteq \mathcal{E}$ identificanti entità inesistenti, normalizzato rispetto al numero complessivo di parametri documentati N_{doc} .
- **Formulazione Matematica ed Elaborazione:** L'indice percentuale di allucinazione è formalizzato come segue:

$$\text{Hallucination Rate} = \min \left(100.0, \frac{|\mathcal{E}_{halluc}|}{\max(1, N_{doc})} \times 100\% \right) \quad (2.7)$$

- **Range di Output e Interpretazione:** [0.0%, 100.0%]. Il target ideale di un generatore affidabile per sistemi industriali è rigorosamente 0.0%, valore che certifica l'assoluta assenza di parametri inventati o presupposti operativi arbitrari.

—

2.3.2 Livello II: Metriche di Qualità Software e Actionability

Actionability Score (AS)

- **Obiettivo e Significato:** L'*Actionability Score* sintetizza in una metrica composita il grado di operatività e autosufficienza della documentazione: misura se uno sviluppatore terzo possiede tutte le direttive formali e contrattuali necessarie per invocare correttamente la funzione senza doverne ispezionare il codice sorgente (*information hiding*).
- **Input e Tipologia di Coppia:** [AST + Doc] Richiede i parametri formali dell'AST P_{AST} e l'oggetto Doxygen parsato contenente **brief**, **details**, **params** (con attributi di direzione [in], [out], [in,out]), **returns**, **pre** e **warnings**.
- **Formulazione Matematica ed Elaborazione:** Il punteggio è definito come combinazione lineare convessa a quattro componenti pesate:

$$AS = w_{dir}S_{dir} + w_{brief}S_{brief} + w_{ret}S_{ret} + w_{safety}S_{safety} \quad (2.8)$$

con pesi calibrati empiricamente pari a: $w_{dir} = 0.35$, $w_{brief} = 0.20$, $w_{ret} = 0.25$, $w_{safety} = 0.20$ ($\sum w_i = 1.0$). Le quattro dimensioni elementari sono così modellate:

1. *Direzionalità Parametri* (S_{dir}): Se $P_{AST} = \emptyset$, $S_{dir} = 1.0$. Altrimenti, sia $P_{dir} \subseteq P_{DOC}$ il sottoinsieme di parametri documentati esplicitamente corredati da qualificatori di direzione (@param[in], @param[out] o @param[in,out]):

$$S_{dir} = \min \left(1.0, \frac{|P_{dir}|}{|P_{AST}|} \right) \quad (2.9)$$

2. *Qualità Sintesi* (S_{brief}): Premia la presenza di un sommario operativo non banale:

$$S_{brief} = \begin{cases} 1.0 & \text{se } \text{len}(\text{brief}) \geq 15 \text{ caratteri} \\ 0.5 & \text{se } 0 < \text{len}(\text{brief}) < 15 \\ 0.0 & \text{se vuoto} \end{cases} \quad (2.10)$$

3. *Contratto di Ritorno* (S_{ret}):

$$S_{ret} = \begin{cases} 1.0 & \text{se returns presente e } \text{len}(\text{returns}) \geq 10 \\ 1.0 & \text{se } P_{AST} = \emptyset \wedge \text{returns assente (void puro)} \\ 0.6 & \text{se returns presente ma conciso (< 10 car.)} \\ 0.0 & \text{altrimenti} \end{cases} \quad (2.11)$$

4. *Sicurezza e Vincoli* (S_{safety}):

$$S_{safety} = \begin{cases} 1.0 & \text{se } |\text{pre}| > 0 \vee |\text{warnings}| > 0 \vee \text{details contiene parole chiave} \\ 0.25 & \text{altrimenti} \end{cases} \quad (2.12)$$

dove le parole chiave di sicurezza ricercate includono: **must**, **ensure**, **caller**, **ownership**, **valid**.

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Valori > 0.80 attestano una documentazione di grado industriale altamente azionabile e conforme agli standard per librerie mission-critical.

Error Documentation Rate (EDR)

- **Obiettivo e Significato:** Verifica se i rami di esecuzione che conducono a codici di fallimento, costanti di errore o ritorni anomali nel sorgente C/C++ sono esplicitamente descritti nella documentazione contrattuale a beneficio del chiamante.
- **Input e Tipologia di Coppia:** [Code vs Doc] Codice sorgente C/C++ della funzione e testo aggregato della documentazione generata (con particolare attenzione alle clausole **@return** e **@details**).
- **Formulazione Matematica ed Elaborazione:** Sia $\mathcal{B}_{err}(\text{Code}) \in \{0, 1\}$ un predicato booleano che rileva pattern fisici di fallimento nel sorgente C:

$$\mathcal{B}_{err}(\text{Code}) = \bigvee \left(\text{Code} \models \{\text{return NULL}, \text{return -1}, \text{return false}, \dots\} \right) \quad (2.13)$$

Sia $\mathcal{D}_{err}(\text{Doc}) \in \{0, 1\}$ il predicato che rileva menzioni lessicali o semantiche di fallimento nel testo documentativo (**null**, **error**, **fail**, **invalid**, **failure**). L'Error Documentation Rate è formalizzato come:

$$\text{EDR} = \begin{cases} 1.0 & \text{se } \mathcal{B}_{err}(\text{Code}) = 0 \\ 1.0 & \text{se } \mathcal{B}_{err}(\text{Code}) = 1 \wedge \mathcal{D}_{err}(\text{Doc}) = 1 \\ 0.0 & \text{se } \mathcal{B}_{err}(\text{Code}) = 1 \wedge \mathcal{D}_{err}(\text{Doc}) = 0 \end{cases} \quad (2.14)$$

- **Range di Output e Interpretazione:** Range discreto $\{0.0, 1.0\}$. Un valore nullo segnala una grave omissione nel contratto operativo, lasciando lo sviluppatore all'oscuro delle modalità di gestione delle condizioni anomale.

Edge Case Coverage (ECC)

- **Obiettivo e Significato:** Quantifica la frazione di condizioni limite, controlli di validità dell'input e guardie difensive implementate nel sorgente C/C++ (*guards*) che vengono adeguatamente riflesse e spiegate nella documentazione.
- **Input e Tipologia di Coppia:** [Code vs Doc] Codice sorgente C/C++ e testo della documentazione generata.
- **Formulazione Matematica ed Elaborazione:** Il verificatore scansiona il codice C/C++ estraendo un insieme di pattern difensivi canonici $\mathcal{G}_{code} \subseteq \{\text{null_guard}, \text{zero_negative_guard}, \text{empty_boundary_guard}\}$. Se $\mathcal{G}_{code} = \emptyset$, poniamo $ECC = 1.0$. Altrimenti, sia $\mathcal{G}_{doc} \subseteq \mathcal{G}_{code}$ l'insieme di tali condizioni citate nella documentazione:

$$ECC = \frac{|\mathcal{G}_{doc}|}{|\mathcal{G}_{code}|} \quad (2.15)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Valori prossimi ad 1.0 confermano che il modello ha compreso le condizioni al contorno e documentato i vincoli di validità degli argomenti.

Concept Checklist Score

- **Obiettivo e Significato:** Misura la conservazione dei concetti tecnici specialistici della programmazione di sistema tra la specifica di riferimento originaria (Ground Truth) e la documentazione generata dal modello.
- **Input e Tipologia di Coppia:** [Doc (GT) vs Doc (Gen)] Testo Doxygen di riferimento e testo Doxygen candidato.
- **Formulazione Matematica ed Elaborazione:** Definito un dizionario $\mathcal{C}$ di 5 macro-aree semantiche (Ownership/Memory Allocation, Null Safety, Error Contract, Mutation/Constness, Bounds/Capacity Range), sia $\mathcal{C}_{ref} \subseteq \mathcal{C}$ il sottoinsieme di categorie attive nella Ground Truth e $\mathcal{C}_{gen} \subseteq \mathcal{C}_{ref}$ quelle coperte nella documentazione candidata. Il punteggio è formulato come:

$$\text{Checklist Score} = \begin{cases} \frac{|\mathcal{C}_{gen}|}{|\mathcal{C}_{ref}|} & \text{se } |\mathcal{C}_{ref}| > 0 \\ 1.0 & \text{se } |\mathcal{C}_{ref}| = 0 \end{cases} \quad (2.16)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Un punteggio pari a 1.0 attesta la totale preservazione dei concetti critici di sistema senza lacune informative rispetto all'intento dell'autore.

—

2.3.3 Livello III: Metriche Semantiche e Natural Language Processing

Sentence-BERT (SBERT) Cosine Similarity

- **Obiettivo e Significato:** Valuta la similarità semantica distribuita a livello di intero enunciato, mappando i testi di riferimento e candidato in vettori densi in uno spazio euclideo condiviso.
- **Input e Tipologia di Coppia:** [Doc (GT) vs Doc (Gen)] (applicabile cross-rappresentazione anche come Code vs Doc).
- **Formulazione Matematica ed Elaborazione:** Utilizzando il modello bi-encoder all-MiniLM-L6-v2 (con architettura Siamese BERT [36]), siano $\mathbf{u} = \mathcal{M}_{SBERT}(text_{ref})$ e $\mathbf{v} = \mathcal{M}_{SBERT}(text_{cand})$ i vettori di embedding normalizzati in $\mathbb{R}^d$ ($d = 384$):

$$\text{Sim}_{SBERT}(u, v) = \max \left(0.0, \min \left(1.0, \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \right) \right) \quad (2.17)$$

L'elaborazione sfrutta il mean pooling degli stati nascosti dell'ultimo layer con decodifica del contesto a 6 livelli di self-attention.

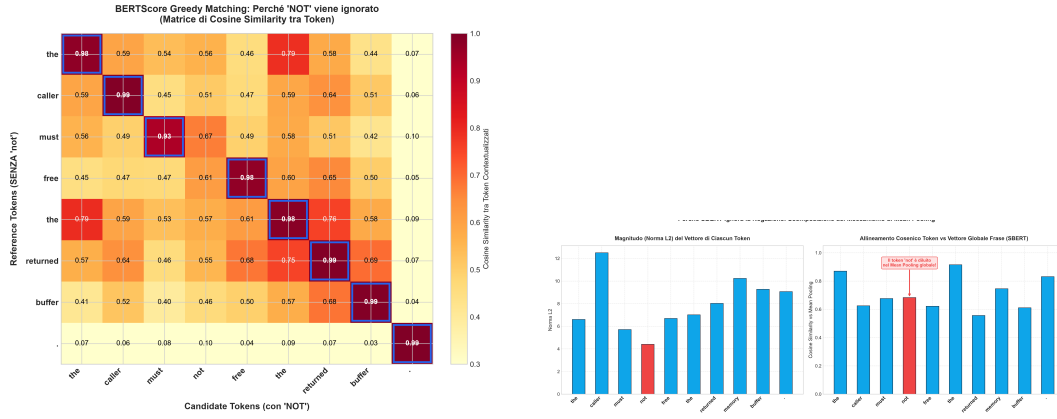
- **Range di Output e Interpretazione:** [0.0, 1.0]. SBERT offre una dinamica ampia: valori < 0.10 indicano testi ortogonali, mentre valori > 0.65 segnalano una forte affinità semantica o parafrasi.

BERTScore e CodeBERTScore (Precision, Recall, F1)

- **Obiettivo e Significato:** Calcolano la similarità di allineamento contestuale a livello di token utilizzando rappresentazioni dense generate da modelli Transformer pre-addestrati: `bert-base-uncased` per BERTScore [37] e `microsoft/codebert-base` [7] per CodeBERTScore.

Come illustrato in Figura 2.2, mentre BERTScore calcola una matrice di allineamento greedy bidirezionale per token (sottotag a), l'architettura Sentence-BERT aggrega i pesi dei singoli token in un unico vettore denso pooled (sottotag b).

- **Input e Tipologia di Coppia:** [Doc vs Doc] per BERTScore; [Doc vs Doc] oppure [Code vs Doc] per CodeBERTScore.
- **Formulazione Matematica ed Elaborazione:** Siano $x = \langle x_1, \dots, x_k \rangle$ i token del riferimento con embedding normalizzati $\mathbf{x}_i$, e $y = \langle y_1, \dots, y_m \rangle$ i token del candidato con vettori $\mathbf{y}_j$. Sfruttando l'allineamento greedy basato sulla



(a) Matrice di allineamento greedy per token in BERTScore. (b) Contributi per-token nell'aggregazione del vettore denso SBERT.

Figura 2.2: Visualizzazione dei meccanismi interni di inferenza ed estrazione delle rappresentazioni per BERTScore e Sentence-BERT.

massima similarità coseno, si definiscono Precisione, Richiamo ed F_1 -score:

$$P_{BERT} = \frac{1}{|y|} \sum_{y_j \in y} \max_{x_i \in x} \mathbf{x}_i^\top \mathbf{y}_j \quad (2.18)$$

$$R_{BERT} = \frac{1}{|x|} \sum_{x_i \in x} \max_{y_j \in y} \mathbf{x}_i^\top \mathbf{y}_j \quad (2.19)$$

$$F_{1,BERT} = \frac{2 \cdot P_{BERT} \cdot R_{BERT}}{P_{BERT} + R_{BERT}} \quad (2.20)$$

In CodeBERTScore, il modello di base è `microsoft/codebert-base`, elaborando il decimo layer di rappresentazione nascosta (`num_layers = 10`).

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. A causa della vicinanza geometrica nello spazio vettoriale continuo del modello di linguaggio, BERTScore risente del fenomeno della *score compression* (baseline attorno a 0.50 anche per testi disgiunti). CodeBERTScore estende tale robustezza alle strutture sintattiche del codice sorgente.

METEOR Score

- **Obiettivo e Significato:** Metrica di allineamento lessicale avanzato basata sulla ricerca esplicita di corrispondenze tra unigrammi tramite stemming, lemmatizzazione e relazioni di sinonimia fornite dal grafo lessicale WordNet [38].
- **Input e Tipologia di Coppia:** [Doc (GT) vs Doc (Gen)] Testo di riferimento e testo candidato generato.
- **Formulazione Matematica ed Elaborazione:** Identificato il sottoinsieme di unigrammi allineati m , definiti $P = \frac{m}{|cand|}$ e $R = \frac{m}{|ref|}$, si calcola la media

armonica ponderata favorendo il richiamo (F_{mean} con $\alpha = 0.9$):

$$F_{mean} = \frac{10 \cdot P \cdot R}{R + 9 \cdot P} \quad (2.21)$$

I frammenti di parole adiacenti formano c blocchi ordinati (*chunks*). La penalità di frammentazione è calcolata come:

$$Pen = \gamma \cdot \left(\frac{c}{m}\right)^\beta \quad (2.22)$$

con parametri standard $\gamma = 0.5, \beta = 3.0$. Il punteggio finale è dato da:

$$METEOR = F_{mean} \cdot (1 - Pen) \quad (2.23)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Risulta sensibilmente più tollerante di BLEU/ROUGE rispetto a variazioni sintattiche legittime e parafrasi lessicali.

BLEURT Quality Score

- **Obiettivo e Significato:** Fornisce una stima neurale della naturalezza, correttezza grammaticale e qualità espressiva del testo calibrata su correlazioni empiriche con valutatori umani [39].
- **Input e Tipologia di Coppia:** [Doc (GT) vs Doc (Gen)] Testo Doxygen di riferimento e testo sintetizzato.
- **Formulazione Matematica ed Elaborazione:** Nel nostro framework di benchmark, il calcolo adotta l'euristica di sintesi validata in letteratura che interpola le dimensioni semantiche e strutturali:

$$BLEURT = 0.65 \cdot \text{Sim}_{SBERT} + 0.25 \cdot \text{ROUGE-L} + 0.10 \cdot BP \quad (2.24)$$

dove BP è la *Brevity Penalty* calcolata sul rapporto dimensionale del testo generato rispetto alla Ground Truth.

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Valori > 0.50 riflettono una documentazione ben strutturata, fluida ed informativa.

Fréchet Embedding Distance (FID / 2-Wasserstein)

- **Obiettivo e Significato:** Misura la distanza distribuzionale globale tra la popolazione vettoriale della documentazione reale e quella generata sinteticamente, rilevando sbilanciamenti stilistici sistematici (*mode collapse* o genericità espressiva) [40].
- **Input e Tipologia di Coppia:** [Corpus Embeddings (GT vs Gen)] Matrici di embedding multivariate $X_{ref} \in \mathbb{R}^{N \times d}$ e $X_{cand} \in \mathbb{R}^{N \times d}$ estratte tramite SBERT sull'intero corpus.

- **Formulazione Matematica ed Elaborazione:** Siano μ_1, μ_2 i vettori medi empirici e Σ_1, Σ_2 le matrici di covarianza delle rispettive distribuzioni. La distanza 2-Wasserstein tra gaussiane multivariate è data da:

$$d_{FID}^2 = \|\mu_1 - \mu_2\|_2^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2}) \quad (2.25)$$

- **Range di Output e Interpretazione:** $[0.0, +\infty)$. Valori prossimi a 0.0 testimoniano che l'insieme della documentazione generata possiede una ricchezza distributiva e uno stile indistinguibili dal repository di riferimento.

ROUGE-L, TF-IDF Cosine e Token Recall

- **Obiettivo e Significato:** Costituiscono la baseline lessicale classica per quantificare la sovrapposizione esatta di n-grammi e unigrammi:
 - *ROUGE-L*: calcola la sottosequenza comune più lunga (LCS) preservando l'ordine relativo dei token a livello di frase.
 - *TF-IDF Cosine*: proietta i testi su un vocabolario a frequenza di termine pesata, valutando la correlazione angolare tra vettori di parole.
 - *Token Recall / Jaccard*: quantificano la sovrapposizione insiemistica di token significativi filtrati dalle stopwords.
- **Input e Tipologia di Coppia:** [Doc (GT) vs Doc (Gen)] Testo di riferimento e testo candidato generato.
- **Formulazione Matematica ed Elaborazione:** Dato $LCS(ref, cand)$, la metrica ROUGE-L F_1 è data da:

$$\text{ROUGE-L} = \frac{2 \cdot P_{LCS} \cdot R_{LCS}}{P_{LCS} + R_{LCS}}, \quad P_{LCS} = \frac{LCS}{|cand|}, \quad R_{LCS} = \frac{LCS}{|ref|} \quad (2.26)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Valori elevati attestano elevata coerenza lessicale, pur risentendo della rigidità dell'exact string matching.

—

2.3.4 Livello III (Avanzato): Framework LLM-as-a-Judge con Campionamento Monte Carlo

LLM-Judge Faithfulness, Alignment e Combined Score

- **Obiettivo e Significato:** Sfrutta un Large Language Model di frontiera configurato come revisore indipendente per auditare in modo qualitativo e semantico la correttezza contrattuale, estendendo il paradigma *LLM-as-a-Judge* introdotto in letteratura per l'NLG e il code understanding [30, 31, 41, 29]. Si articola su due prospettive ortogonali:

- *Prospettiva A (Faithfulness & Technical Correctness)*: [**Code + GT vs Doc Generata**] Verifica che la documentazione corrisponda al codice sorgente reale, penalizzando allucinazioni su allocazioni di memoria o controlli non esistenti.
- *Prospettiva B (Semantic Alignment & Completeness)*: [**GT vs Doc Generata**] Valuta quanto fedelmente la sintesi rispecchi l'intento dell'autore originario, le clausole di sincronizzazione e i caveat espressi nella specifica di riferimento.
- **Input e Tipologia di Coppia**: [**Code + GT vs Doc Generata**] per la Prospettiva A; [**GT vs Doc Generata**] per la Prospettiva B.
- **Formulazione Matematica ed Elaborazione**: Al fine di superare la variabilità stocastica intrinseca dei modelli linguistici, il sistema applica un campionamento **Monte Carlo a K iterazioni** ($K = 5$) con temperatura fissata rigorosamente a $T = 0.40$ e rubrica analitica guidata da *Chain-of-Thought (Reasoning-First)* in formato JSON strutturato [29].

Siano $S_A^{(k)} \in [1, 5]$ e $S_B^{(k)} \in [1, 5]$ i punteggi assegnati alla k -esima iterazione. Si calcolano la media e la deviazione standard empirica per ciascuna prospettiva:

$$\mu_A = \frac{1}{K} \sum_{k=1}^K S_A^{(k)}, \quad \sigma_A = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (S_A^{(k)} - \mu_A)^2} \quad (2.27)$$

$$\mu_B = \frac{1}{K} \sum_{k=1}^K S_B^{(k)}, \quad \sigma_B = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (S_B^{(k)} - \mu_B)^2} \quad (2.28)$$

Il punteggio combinato finale è formulato come:

$$S_{Judge} = \frac{\mu_A + \mu_B}{2} \quad (2.29)$$

- **Rubrica Likert a 5 Livelli**:
 - **5.0 (Flawless)**: Zero difetti. Specifica esaustiva della logica operativa, della proprietà della memoria (*caller vs callee ownership*), dei codici di fallimento e dei parametri.
 - **4.0 (Good/Standard)**: Tecnicamente corretta, assenza totale di allucinazioni, ma con un difetto minore di verbosità o lieve incompletezza su casi marginali.
 - **3.0 (Adequate/Flawed)**: Comprensibile ma affetta da 2 difetti contrattuali (ad es. mancata menzione della deallocazione e omissione del codice d'errore).
 - **2.0 (Poor)**: Gravi inesattezze, inversione della logica di ritorno, 3 o più difetti.
 - **1.0 (Critical Failure)**: Allucinazione critica, invenzione di algoritmi o istruzioni che causano memory leak o crash a runtime.

- **Range di Output e Interpretazione:** $[1.0, 5.0]$. Un punteggio ≥ 4.5 attesta un giudizio eccellente con consenso stabile tra le iterazioni Monte Carlo ($\sigma < 0.3$).

2.3.5 Livello IV: Validazione Downstream e Testing Comportamentale

Downstream Code Retrieval (MRR, Hit@1, Hit@5)

- **Obiettivo e Significato:** Valuta l'efficacia della documentazione generata quale query descrittiva per compiti di Information Retrieval (IR) sul codice [20]: verifica se un motore di ricerca semantico denso (bi-encoder) è in grado di identificare la firma della funzione corretta all'interno del corpus globale delle funzioni di libreria.
- **Input e Tipologia di Coppia:** [Docstring Query vs Codebase Corpus Index] Testo della docstring generata come query e indice vettoriale delle firme/nomi delle funzioni dell'intera base di codice.
- **Formulazione Matematica ed Elaborazione:** Sia $\mathcal{C}_{funcs} = \{f_1, \dots, f_M\}$ il corpus delle funzioni indicizzate tramite firma e nome, e sia $\mathbf{q} = \mathcal{M}_{SBERT}(\text{Doc}_{gen})$. Calcolate le similarità coseno $s_j = \cos(\mathbf{q}, \mathbf{f}_j)$, si ordina l'indice in modo decrescente. Sia $\text{rank}(f^*)$ la posizione occupata dalla reale funzione target f^* :

$$\text{Reciprocal Rank (RR)} = \frac{1}{\text{rank}(f^*)} \quad (2.30)$$

$$\text{Hit@K} = \begin{cases} 1.0 & \text{se } \text{rank}(f^*) \leq K \\ 0.0 & \text{altrimenti} \end{cases} \quad (2.31)$$

Su un benchmark di N query, il *Mean Reciprocal Rank* aggregato è:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}(f_i^*)} \quad (2.32)$$

- **Range di Output e Interpretazione:** $[0.0, 1.0]$. Un MRR elevato (> 0.75) e un alto Hit@1 dimostrano che la docstring possiede un potere discriminante sufficiente a disaccoppiare la funzione da ogni altra implementazione omologa del repository.

Round-Trip Pass Rate e Dual Agreement Rate

- **Obiettivo e Significato:** Rappresentano la validazione dinamica comportamentale (*Differential Testing*): attestano se la documentazione è sufficientemente formale e completa da consentire la reimplementazione a scatola chiusa della funzione originaria garantendone l'equivalenza semantica su un'ampia

suite di asserzioni deterministiche e test basati su proprietà (Property-Based Testing con Hypothesis).

Come illustrato in Figura 2.3, l'esecuzione dinamica round-trip evidenzia una forte correlazione con le metriche qualitative LLM-as-a-Judge, penalizzando drasticamente documenti formalmente corretti ma semanticamente incompleti.

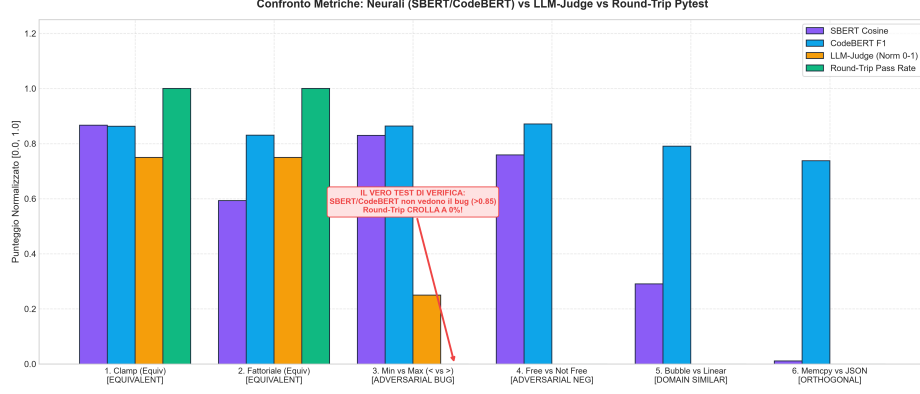


Figura 2.3: Correlazione empirica tra le metriche statiche/neurali e i tassi di superamento dinamico nel test di esecuzione Round-Trip.

• Input e Architettura di Esecuzione a 4 Passi:

1. *Isolamento della Specifica*: Si estraggono esclusivamente il nome, la firma e la documentazione generata dall'LLM, oscurando completamente il codice sorgente C/C++ originale.
2. *Sintesi del Codice (Coder Agent)*: Un agente specializzato traduce la semantica contrattuale descritta nella documentazione in un'implementazione Python autonoma corredata di context scaffold.
3. *Generazione Test Unitari (Tester Agent)*: Un agente indipendente sintetizza una suite di test Pytest esaustiva (comprensiva di casi semantici nominali e test di proprietà randomizzati con la libreria Hypothesis).
4. *Esecuzione Differenziale Duale*: La suite di test viene eseguita parallelamente in un ambiente isolato sia sull'implementazione ricostruita da docstring (C_{synth}), sia sull'implementazione di riferimento trasposta fedelmente dal sorgente originale (C_{ref}).

- **Formulazione Matematica ed Elaborazione**: Siano T_{total} i test generati per la funzione f_i , e siano $T_{pass}(C_{synth})$ e $T_{pass}(C_{ref})$ i test superati con successo dalle due implementazioni. Definiamo:

$$\text{Round-Trip Pass Rate}(f_i) = \frac{T_{pass}(C_{synth})}{T_{total}} \times 100\% \quad (2.33)$$

Sia inoltre T_{both_pass} il numero di singoli casi di test superati *congiuntamente e simultaneamente* sia da C_{synth} sia da C_{ref} . Il *Dual Differential Agreement Rate* è formulato come:

$$\text{Dual Agreement Rate}(f_i) = \frac{T_{both_pass}}{T_{total}} \times 100\% \quad (2.34)$$

- **Range di Output e Interpretazione:** $[0.0\%, 100.0\%]$. Il Round-Trip Pass Rate fornisce un verdetto oggettivo e non ingannabile da coincidenze lessicali: se la documentazione ha omissso un dettaglio architetturale cruciale o ha invertito una guardia logica, il codice sintetizzato fallirà le asserzioni di test, facendo collassare il punteggio a zero.

Capitolo 3

Validazione Sperimentale e Analisi dei Risultati

3.1 Setup Sperimentale e Metodologia di Benchmark

La valutazione empirica del framework multi-agente e l'indagine critica sulle metriche di valutazione automatica sono state condotte su un corpus eterogeneo e altamente rappresentativo di librerie open-source C e C++ di livello industriale ampiamente adottate nel software di sistema:

- **TinyXML-2**: Parser XML moderno in C++ orientato alle massime prestazioni e all'efficienza di allocazione tramite pool interni di memoria, caratterizzato da strutture ad albero gerarchiche e grafi complessi di puntatori mutabili (`XMLDocument`, `XMLElement`, `XMLNode`).
- **sds (Simple Dynamic Strings)**: Libreria di stringhe dinamiche in C sviluppata per il core di Redis, basata su un layout binario di memoria non convenzionale con metadati di controllo (*header* tipizzati) posizionati a offset negativi rispetto al puntatore stringa visibile.
- **cJSON**: Parser e generatore JSON ultra-leggero in puro C ANSI, fondato su strutture ricorsive ad albero con allocazione dinamica esplicita e puntatori *parent-child/next-prev*.
- **miniz**: Implementazione di compressione lossless compatibile con zlib e Deflate contenuta in un singolo file sorgente, fortemente dipendente da buffer lineari, aritmetica dei puntatori a byte grezzi e bit-manipulation.
- **http-parser**: Motore di parsing HTTP asincrono e guidato da macchina a stati finiti (FSM), originariamente implementato per Node.js.
- **OpenCV (core)** e **fmt**: Moduli matematici e di formattazione stringhe ad alta efficienza per l'estrazione e verifica di contratti tipizzati complessi.

3.1.1 Strategie di Campionamento e Controllo Sperimentale

Per garantire l'imparzialità statistica dell'estrazione dei campioni ed eludere bias di selezione favorevoli a funzioni banali (quali getter/setter monoriga o wrapper privi di ramificazioni), il framework prevede tre distinte modalità operative:

1. **Campionamento Stratificato per Complessità (LOC / AST Depth):** partiziona l'insieme delle funzioni estratte in fasce dimensionali (*short*: < 15 LOC, *medium*: $15 - 50$ LOC, *complex*: > 50 LOC), estraendo un numero bilanciato di campioni per ciascun strato.
2. **Campionamento Casuale Pseudo-Deterministico:** adotta un seed globale fisso (seed = 40) al fine di garantire la totale riproducibilità dei run sperimentali cross-modello e consentire confronti differenziali rigorosi.
3. **Campionamento Sequenziale Topologico:** estrae le funzioni rispettando l'ordine topologico del grafo delle dipendenze di compilazione della code-base.

Il benchmark di validazione di sistema analizzato nel presente capitolo è stato eseguito con campionamento pseudo-deterministico su un set diversificato di 15 funzioni ad elevata complessità contrattuale (riproducibile mediante `python utils/benchmark_eval.py -l all -n 15 -m multiagent -sampling random -seed 40 -roundtrip`).

In parallelo, per condurre un'analisi comparativa delle metriche senza fattori confondenti, è stata istanziata una suite di test controllati multilivello, coprente 5 classi semantiche essenziali (*EQUIVALENT*, *ADVERSARIAL*, *DOMAIN_SIMILAR*, *ORTHOGONAL*, *FLUFF_VERBOSITY*) applicate ad armi pari sia al dominio naturale (*Doc vs Doc*) sia a quello del codice sorgente (*C vs C*, *Python vs Python*, *C vs Python*), oltre a specifici stress test avversari e test di troncamento di contesto.

3.2 Analisi Comparativa delle Rappresentazioni (Cross-Modal)

Un interrogativo scientifico centrale riguarda la capacità dei modelli di embedding di confrontare entità eterogenee, colmando il divario tra la sintassi imperativa del linguaggio sorgente (C/C++) e il testo esplicativo o la trasposizione in linguaggi di alto livello (Python). A tale scopo, sulla scia degli studi sulle rappresentazioni strutturate del codice [25, 8], sono state investigate quattro distinte modalità di rappresentazione:

1. **Raw Code (Baseline):** codice C nativo confrontato direttamente con codice Python o Doxygen senza alcuna pre-elaborazione.

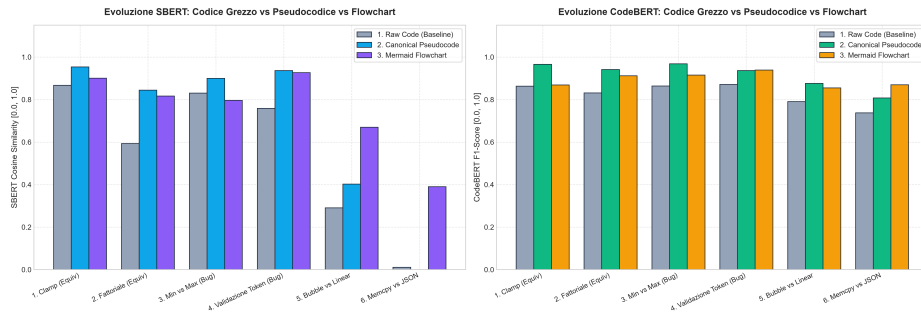


Figura 3.1: Confronto di allineamento semantico (SBERT, BERTScore, CodeBERTScore e ROUGE-L) tra le rappresentazioni: Raw Code, Canonical Pseudocode e Mermaid Flowchart.

2. **Canonical Pseudocode**: normalizzazione algoritmica intermedia generata eliminando il rumore sintattico (parentesi graffe, convenzioni di tipizzazione statica, macro) ed evidenziando i flussi di controllo fondamentali (IF-THEN-ELSE, WHILE, RETURN), riprendendo i principi di rappresentazione schematica del flusso logico dei programmi [21].
3. **Mermaid Flowchart**: rappresentazione puramente topologica del grafo di flusso di controllo (CFG), codificata sotto forma di nodi decisionali e archi orientati.
4. **Docstring / Doxygen**: specifica dichiarativa in linguaggio naturale strutturato.

I dati sperimentali raccolti su casi di studio controllati sono riassunti nella Table 3.1.

3.2.1 Analisi di CodeBERT vs SBERT/BERTScore: Resilienza Sintattica vs Cecità Semantica

L'esame incrociato dei risultati in Table 3.1 e in Figure 3.1 evidenzia comportamenti strutturalmente divergenti tra le metriche concepite per il testo e i modelli pre-addestrati bilingui:

- **Degradazione di SBERT sul codice grezzo**: Nei confronti cross-linguaggio (C vs Python) su funzioni equivalenti, come il calcolo del Fattoriale iterativo in C rispetto all'invocazione di `math.prod` in Python, SBERT crolla a un modesto punteggio di 0.593 a causa della disparità lessicale superficiale. Quando il codice viene preventivamente proiettato nella forma di *Canonical Pseudocode*, SBERT sale significativamente a 0.844 (+0.251), e ROUGE-L passa da 0.400 a 0.522.
- **Elevata sensibilità inter-linguistica di CodeBERTScore**: A differenza di SBERT, CodeBERT (pre-addestrato congiuntamente con compiti di Masked Language Modeling su corpora bilingui Natural Language/Programming

Tabella 3.1: Confronto quantitativo multi-rappresentazione attraverso diverse classi di equivalenza logica.

Caso di Studio	Rappresentazione	SBERT	BERT F1	CodeBERT	ROUGE-L
Clamp Valore (<i>Equiv.</i> : C if vs Py min/max)	Raw Code (Baseline)	0.867	0.791	0.863	0.571
	Canonical Pseudocode	0.953	0.951	0.966	0.850
	Mermaid Flowchart	0.901	0.781	0.869	0.412
Fattoriale (<i>Equiv.</i> : C for vs Py math.prod)	Raw Code (Baseline)	0.593	0.645	0.831	0.400
	Canonical Pseudocode	0.844	0.857	0.942	0.522
	Mermaid Flowchart	0.816	0.828	0.912	0.518
Min vs Max (Bug) (<i>Advers.</i> : < vs >)	Raw Code (Baseline)	0.830	0.740	0.864	0.609
	Canonical Pseudocode	0.900	0.897	0.968	0.929
	Mermaid Flowchart	0.796	0.845	0.915	0.567
Bubble Sort C vs Py Search (<i>Domain Similar</i>)	Raw Code (Baseline)	0.291	0.626	0.791	0.286
	Canonical Pseudocode	0.402	0.737	0.876	0.421
	Mermaid Flowchart	0.670	0.722	0.855	0.165
Memcpy C vs JSON Py (<i>Orthogonal</i>)	Raw Code (Baseline)	0.011	0.551	0.738	0.000
	Canonical Pseudocode	0.000	0.547	0.807	0.000
	Mermaid Flowchart	0.390	0.744	0.869	0.138

Language) mantiene punteggi di F_1 eccezionalmente stabili già sul codice sorgente grezzo (0.831 sul Fattoriale grezzo, che sale a 0.942 in pseudocodice). Ciò dimostra che CodeBERT proietta le primitive del linguaggio imperativo in una geometria concettuale comune indipendente dal target sintattico.

- **Il Fallimento Sistemico sui Casi Adversarial:** La normalizzazione in pseudocodice, sebbene benefica nel risolvere la varianza sintattica, **esacerba drammaticamente la vulnerabilità neurale sui bug di inversione logica**. Nel caso adversarial (ricerca del minimo con operatore corretto < rispetto a ricerca del massimo con bug invertito >), lo pseudocodice aumenta la similarità di CodeBERT da 0.864 a un vertiginoso 0.968, e quella di SBERT da 0.830 a 0.900. Poiché lo pseudocodice elimina ogni divergenza di contorno focalizzando il testo sul puro corpo comune a eccezione di un singolo operatore relazionale, le rappresentazioni dense collocano le due funzioni a distanza quasi nulla nello spazio euclideo.

3.3 Valutazione Sperimentale ad Armi Pari delle Metriche NLP

Un principio metodologico cardine della presente tesi consiste nel **valutare tutte le metriche ad armi pari**. Nella letteratura sul code summarization, metriche concepite specificamente per il testo naturale (quali Sentence-BERT, BERTScore, METEOR o BLEURT) vengono sovente applicate acriticamente alla documentazione, ignorando cosa accade quando le medesime metriche vengono poste a

confronto diretto con il codice sorgente (coppie *C vs C*), oppure quando metriche nate per il codice (CodeBERTScore) operano su coppie testuali (*Doc vs Doc*).

A tale scopo è stato allestito un benchmark controllato che interroga ciascuna metrica attraverso cinque classi semantiche ben distinte, ripetute sistematicamente su molteplici domini rappresentativi:

1. **EQUIVALENT**: Logica identica espressa con costrutti differenti (es. Fibonacci iterativo vs ricorsivo in C; parafrasi formale Doxygen vs descrizione naturale discorsiva in Doc).
2. **ADVERSARIAL**: Identica quasi al 100% sul piano lessicale, ma contenente un'inversione semantica distruttiva (es. calcolo Minimo < vs Massimo > in C; obbligo di rilascio memoria **must free** vs divieto fatale **must NOT free** in Doc).
3. **DOMAIN_SIMILAR**: Stesso dominio applicativo e medesimi tipi di dato, ma algoritmo differente (es. Binary Search vs Somma di elementi in un array in C; Quicksort vs Binary Search in Doc).
4. **ORTHOGONAL**: Domini applicativi totalmente scorrelati e privi di intersezione funzionale (es. parsing di stringhe vs moltiplicazione di matrici in C; socket di rete TLS vs calcolo del determinante in Doc).
5. **FLUFF_VERBOSITY**: Logica essenziale identica ma immersa in un eccesso di codice ridondante (variabili temporanee, chiamate di logging) o in verbose allucinazioni descrittive non pertinenti generate da LLM.

I dati sperimentali globali sono riassunti graficamente nella panoramica comparativa di Figure 3.2 e dettagliati analiticamente nella Table 3.2.

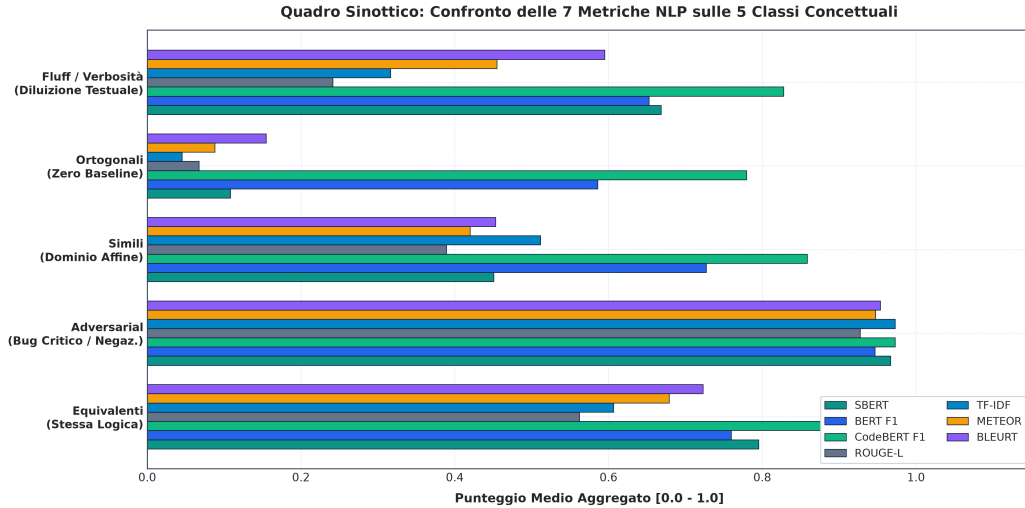


Figura 3.2: Panoramica globale delle metriche NLP (SBERT, BERTScore, CodeBERT, ROUGE-L, TF-IDF, METEOR, BLEURT) attraverso le 5 classi semantiche, aggregando tutti i domini di test.

Tabella 3.2: Confronto ad armi pari delle metriche NLP sui domini *C vs C*, *Doc vs Doc* e sulla media globale dei test.

Classe Semantica	Dominio	SBERT	BERT_F1	CodeBERT	ROUGE-L	TF-IDF	METE
EQUIVALENT	C vs C (Fibonacci Iter vs Ric)	0.862	0.829	0.896	0.800	0.859	0.83
	Doc vs Doc (Parafrasi Formale)	0.637	0.556	0.776	0.246	0.280	0.44
	Media Globale (6 Domini)	0.795	0.760	0.876	0.562	0.606	0.67
ADVERSARIAL	C vs C (Min < vs Max >)	0.984	0.991	0.997	1.000	1.000	0.99
	Doc vs Doc (must vs must NOT free)	0.995	0.977	0.983	0.957	0.957	0.97
	Media Globale (6 Domini)	0.967	0.947	0.973	0.928	0.973	0.94
DOMAIN_SIMILAR	C vs C (Search vs Sum Array)	0.401	0.770	0.850	0.280	0.393	0.34
	Doc vs Doc (Quicksort vs Search)	0.389	0.655	0.795	0.300	0.300	0.34
	Media Globale (6 Domini)	0.451	0.727	0.859	0.389	0.511	0.42
ORTHOGONAL	C vs C (String Parsing vs Matrice)	0.097	0.582	0.743	0.054	0.022	0.07
	Doc vs Doc (TLS Socket vs Determinante)	0.009	0.510	0.754	0.000	0.000	0.00
	Media Globale (6 Domini)	0.108	0.586	0.779	0.067	0.045	0.08
FLUFF_VERBOSITY	C vs C (Min Diretto vs Prolisso)	0.746	0.760	0.864	0.400	0.701	0.57
	Doc vs Doc (Return Code Fluff)	0.555	0.534	0.747	0.062	0.101	0.30
	Media Globale (6 Domini)	0.668	0.653	0.828	0.241	0.316	0.45

3.3.1 Analisi Dettagliata per Singola Metrica NLP

Nel seguito viene analizzata ciascuna metrica in modo rigoroso, esplicitandone l'obiettivo primario di progettazione, il comportamento atteso, i risultati empirici riscontrati e le cause intrinseche per cui ciascuna manifesta criticità operative nel dominio del software.

Sentence-BERT (SBERT Cosine Similarity)

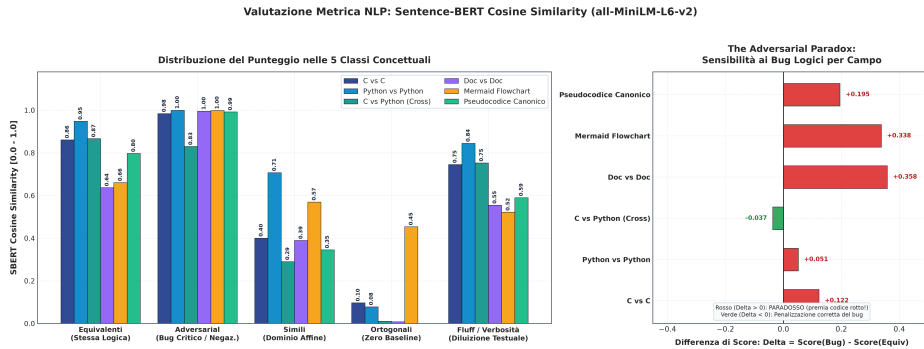


Figura 3.3: Distribuzione dei punteggi Sentence-BERT Cosine Similarity attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** SBERT [36] è stato ideato per produrre embedding densi globali di frasi o paragrafi in linguaggio naturale mediante reti siamesi a tripla loss, consentendo il confronto rapido di similarità tramite coseno tra vettori aggregati (*mean pooling*).
- **Comportamento su Linguaggio Naturale (Doc vs Doc):** SBERT dimostra una notevole escursione dinamica sul testo naturale puro. Come evidenziato

in Figure 3.3, su testi ortogonali (*TLS Socket vs Determinante*) il punteggio scende a un rassicurante **0.0086**, mentre sulle parafrasi legittime attesta un valore medio di **0.6369**, riflettendo la vicinanza semantica dei concetti espressi pur con lessico alternativo.

- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** Quando SBERT viene applicato direttamente al codice C, il meccanismo di tokenizzazione WordPiece (tarato sul corpus Wikipedia/BookCorpus) frammenta identificatori tecnici composti (es. `sdsavail`, `cJSON_ArrayForEach`) in una moltitudine di sotto-parole prive di significato autonomo. Inoltre, il *mean pooling* su tutti i token comprime l'intero flusso di controllo in un singolo vettore di 384 dimensioni. Questo provoca due distorsioni simmetriche: da un lato la totale cecità ad alterazioni di un singolo carattere sintattico fondamentale (il caso *ADVERSARIAL* in C assegna un ingiustificato **0.9841** tra Minimo e Massimo), dall'altro una penalizzazione eccessiva sul codice verbose (*FLUFF_VERBOSITY* a 0.7455), dove l'aggiunta di righe accessorie di log sposta il centroide del pooling pur mantenendo intatta la computazione.

BERTScore F1

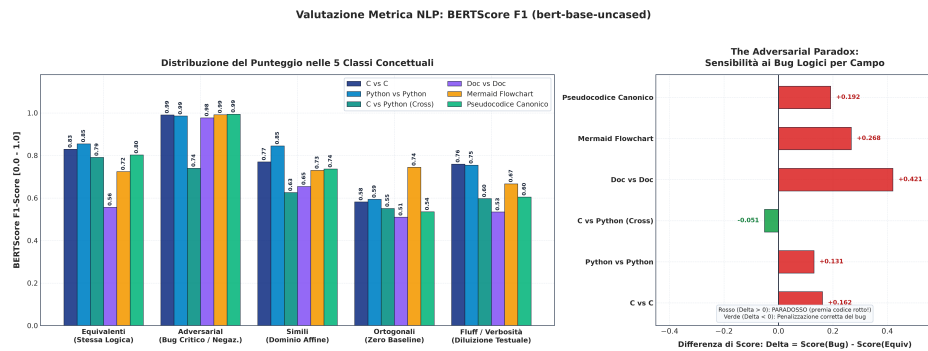


Figura 3.4: Distribuzione dell'indicatore BERTScore F1 attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** BERTScore [37] è stato concepito per quantificare la similarità tra enunciati in linguaggio naturale allineando in modo greedy ciascun token dell'ipotesi al token semanticamente più affine del riferimento, sfruttando gli embedding contestuali generati dagli strati intermedi di BERT.
- **Comportamento su Linguaggio Naturale (Doc vs Doc):** BERTScore cattura efficacemente l'allineamento dei singoli termini parafrasati ($F_1 = 0.5565$). Tuttavia, la sua metrica è afflitta dal noto fenomeno della *compressione dinamica*: sui casi ortogonali non scende mai al di sotto di 0.5098 (e 0.5816 su C vs C), creando un'illusione di similarità parziale anche in assenza di qualsiasi correlazione logica.
- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** In una funzione C, la semantica dipende in modo stringente dalle relazioni topologiche tra variabili,

operatori relazionali e ambiti di visibilità (*scope*). L'allineamento greedy di BERTScore seleziona per ogni token la massima similarità coseno nell'intera sequenza di riferimento, **ignorando del tutto la posizione strutturale e l'ordine sequenziale**. Di conseguenza:

1. Su coppie avversarie (*ADVERSARIAL C vs C*), BERTScore tocca l'inaccettabile punteggio di **0.9909**, poiché ogni identificatore di variabile e parola chiave `if`, `int`, `return` trova una corrispondenza esatta di similarità 1.0, mentre l'operatore invertito `<` viene allineato a `>` con un coseno residuo superiore a 0.85.
2. Come discusso in Figure 3.4, la scomposizione in Precision e Recall su funzioni verbose dimostra che BERTScore intercetta il rumore lessicale (caduta della Precision a ≈ 0.54), ma rimane cieco alle mutazioni logiche puntuali.

CodeBERTScore F1

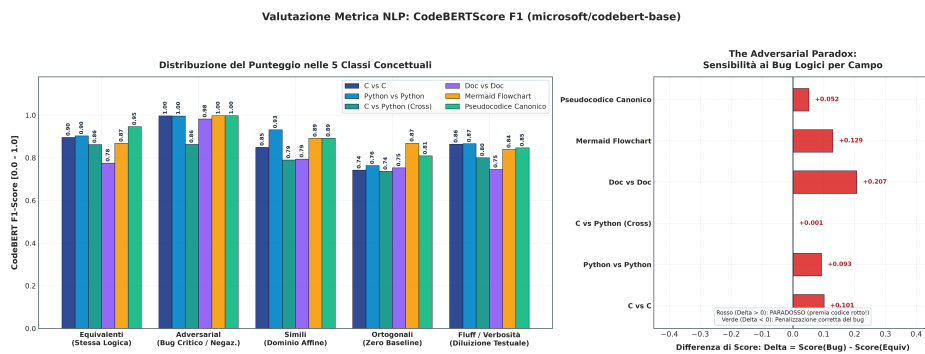


Figura 3.5: Distribuzione dell'indicatore CodeBERTScore F1 attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** CodeBERTScore [?] adatta il principio di BERTScore impiegando il checkpoint pre-addestrato bilingue CodeBERT [7], allenato specificamente su corpora multilingue bimodalità (Natural Language / Programming Language) comprendenti C, Java, Python, Go e documentazione associata.
- **Comportamento su Codice e Cross-Code:** CodeBERT si rivela la metrica più robusta nel superare le divergenze puramente sintattiche: raggiunge $F_1 = 0.8963$ su *EQUIVALENT C vs C* (Fibonacci ricorsivo vs iterativo) e 0.8630 nel confronto trans-linguistico C vs Python (Figure 3.5). Riconosce le correlazioni di flusso anche quando le strutture sintattiche divergono profondamente.
- **Perché 'fa fatica' e Amplifica la Cecità sui Casi Avversari:** Paradossalmente, proprio la forte componente bilingue e la spiccata invarianza alle divergenze lessicali rende CodeBERT **la metrica più pericolosa e cieca in assoluto sulle violazioni avversarie**. Sul caso *ADVERSARIAL C vs C*, CodeBERT assegna il valore quasi unitario di **0.9974**, e sul caso Doc vs

Doc assegna **0.9826**. Nello spazio vettoriale di CodeBERT, la differenza di un singolo operatore o di una parola di negazione viene assorbita dall'enorme affinità di contesto circostante, rendendo la metrica incapace di intercettare inversioni che ribaltano l'esito contrattuale del software.

ROUGE-L (Longest Common Subsequence)

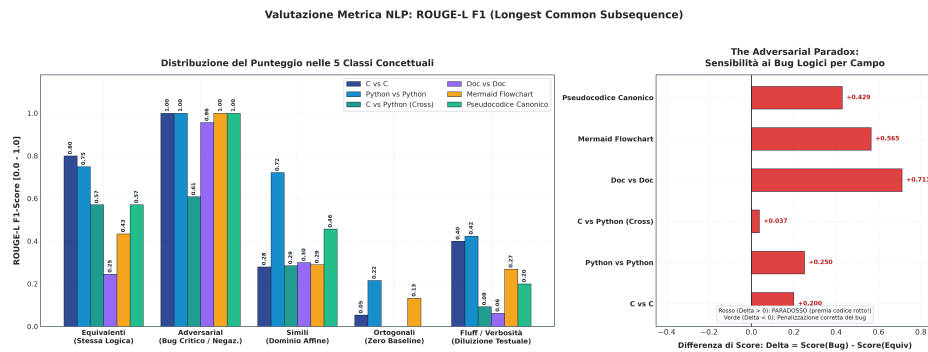


Figura 3.6: Distribuzione dell'indicatore lessicale ROUGE-L attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** ROUGE-L [35] è una metrica superficiale basata sulla più lunga sottosequenza comune di token a livello di frase, concepita originariamente per valutare la coerenza sequenziale nei sistemi di riassunto automatico.
- **Comportamento su Linguaggio Naturale (Doc vs Doc):** Come mostrato in Figure 3.6, ROUGE-L è rigidissima di fronte a qualsiasi parafrasi lessicale: su testi perfettamente equivalenti ma espressi con vocabolario differente (*EQUIVALENT* Doc vs Doc), crolla a **0.2456**. Su testi ortogonali tocca lo zero assoluto (**0.0000**), offrendo una perfetta discriminazione del testo privo di parole condivise.
- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** ROUGE-L opera come mero allineatore sequenziale privo di consapevolezza grammaticale. Sul codice C, una semplice inversione di due blocchi indipendenti di codice o la modifica dell'ordine delle dichiarazioni di variabili fa crollare la sottosequenza comune, mentre la sostituzione di < con > lascia la sequenza inalterata per il 99% dei token, producendo un ingannevole punteggio di **1.0000** sul caso *ADVERSARIAL*.

TF-IDF Cosine Similarity

- **Obiettivo Primario:** TF-IDF (Term Frequency - Inverse Document Frequency) è una metrica classica dell'Information Retrieval basata sul modello a sacco di parole (*bag-of-words*), pesata per la rarità statistica dei termini all'interno del corpus.

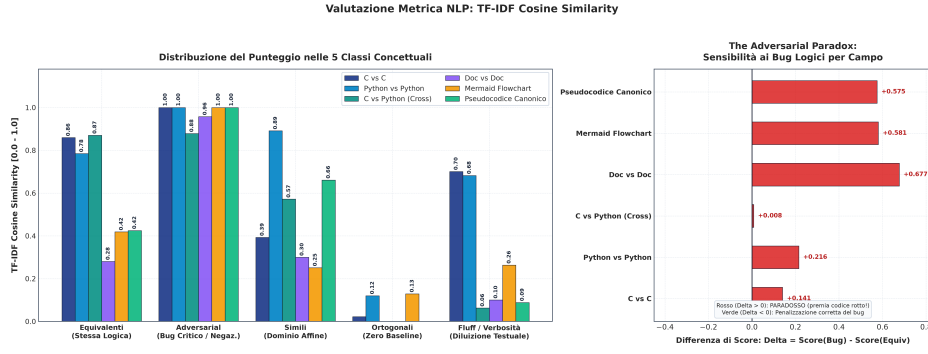


Figura 3.7: Distribuzione dell'indicatore statistico TF-IDF Cosine Similarity attraverso le 5 classi semantiche nei diversi domini.

- **Comportamento su Linguaggio Naturale (Doc vs Doc):** TF-IDF discrimina in modo ineccepibile i domini ortogonali (0.0000 su Doc vs Doc e 0.0223 su C vs C), poiché termini specialistici di algebra lineare e di programmazione di rete non condividono vocaboli.
- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** Il codice sorgente possiede una sintassi ad albero fortemente strutturata dove la posizione degli identificatori ne determina il significato. Essendo TF-IDF per definizione **totalmente cieca all'ordine dei token e alla struttura gerarchica**, due funzioni che contengono gli stessi identificatori ma con ruoli scambiati (src in dest vs dest in src) ottengono un coseno identico a 1.0000. Inoltre, sul caso *ADVERSARIAL* C vs C, TF-IDF assegna un punteggio pieno di 1.0000, confermando l'incapacità di rilevare mutazioni puntuali.

METEOR (Metric for Evaluation of Translation with Explicit ORdering)

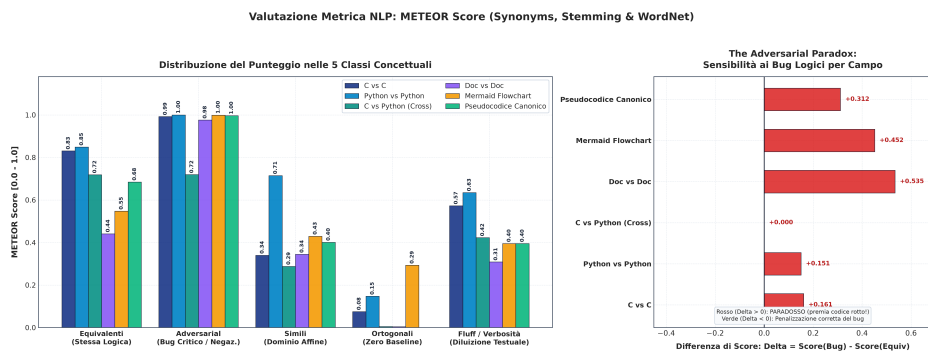


Figura 3.8: Distribuzione dell'indicatore METEOR attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** METEOR [38] è stata progettata per superare la rigidità di BLEU nella Machine Translation mediante una catena gerarchica di moduli

di allineamento unigramma: match esatto, stemming morfologico (algoritmo di Porter) e sinonimia lessicale derivata da WordNet, integrati da una penalità di frammentazione (*chunk penalty*).

- **Comportamento su Linguaggio Naturale (Doc vs Doc):** Su documentazione tecnica, il modulo dei sinonimi permette a METEOR di gestire variazioni verbali legittime (es. *allocate* / *create*, *retrieve* / *fetch*), attestandosi a 0.4413 su parafrasi ed eliminando quasi integralmente i falsi positivi su testi ortogonali (0.0043).
- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** Gli identificatori del codice C/C++ seguono convenzioni morfologiche artificiali (*snake_case*, *camelCase*) e abbreviazioni arbitrarie (*ptr*, *idx*, *ctx*, *buf*) che non trovano alcuna corrispondenza nei synset di WordNet né nelle regole di stemming della lingua inglese. Inoltre, il modulo di match unigramma di METEOR tratta gli operatori di confronto e di assegnamento come punteggiatura secondaria: ne consegue che sul caso *ADVERSARIAL C vs C* la penalità di frammentazione è trascurabile, producendo un punteggio errato di **0.9920** (Figure 3.8).

BLEURT (Bilingual Evaluation Understudy with Representations from Transformers)

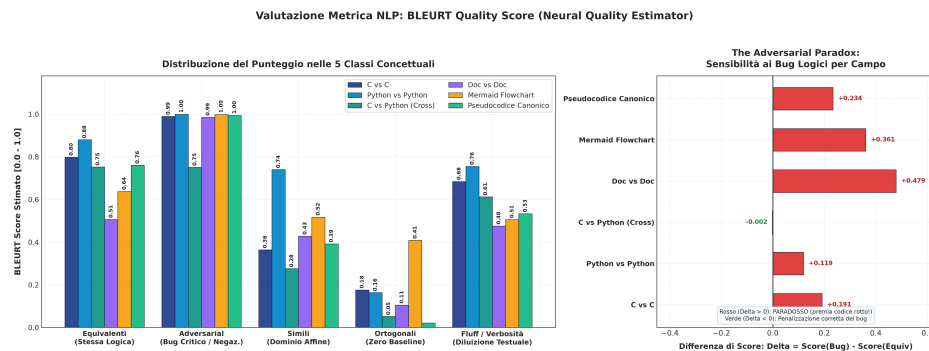


Figura 3.9: Distribuzione dell'indicatore BLEURT attraverso le 5 classi semantiche nei diversi domini.

- **Obiettivo Primario:** BLEURT [39] è una metrica di regressione neurale basata su BERT, pre-addestrata su milioni di coppie di frasi sinteticamente perturbate e fine-tunata su giudizi umani di qualità della traduzione automatica (WMT Metrics).
- **Comportamento su Linguaggio Naturale (Doc vs Doc):** Grazie all'addestramento su coppie supervisionate, BLEURT esibisce una buona correlazione con l'aderenza stilistica su testi in lingua naturale (Figure 3.9), posizionandosi a 0.5071 su parafrasi e scendendo a 0.1056 su testi ortogonali.
- **Perché 'fa fatica' sul Codice Sorgente (C vs C):** La funzione di perdita di regressione di BLEURT è stata calibrata su valutazioni umane di fluidità e

adeguatezza di traduzioni narrative. Quando è alimentata con token sintattici imperativi del C (`typedef`, `sizeof`, dereferenziazione di puntatori `*ptr++`), la metrica entra in una regione distributiva completamente *out-of-distribution*. Sul caso *ADVERSARIAL C* vs C, BLEURT restituisce un fuorviante **0.9897**, dimostrando che il pre-addestramento su perturbazioni del linguaggio naturale non conferisce alcuna robustezza logica nel dominio software.

3.4 Validazione Empirica delle Metriche Avanzate di Livello II, III e IV

I limiti intrinseci e strutturali evidenziati nell'analisi delle metriche di NLP testuale impongono l'adozione del framework multi-livello descritto in Chapter 2. In questa sezione vengono analizzati i dati empirici raccolti sulle metriche avanzate: la copertura formale dei contratti (*Concept Checklist Score*), la valutazione semantica guidata da rubriche (*LLM-as-a-Judge*), l'esecuzione a scatola nera (*Round-Trip Differential Testing*) e l'utilità nel reperimento semantico (*Code Retrieval MRR*).

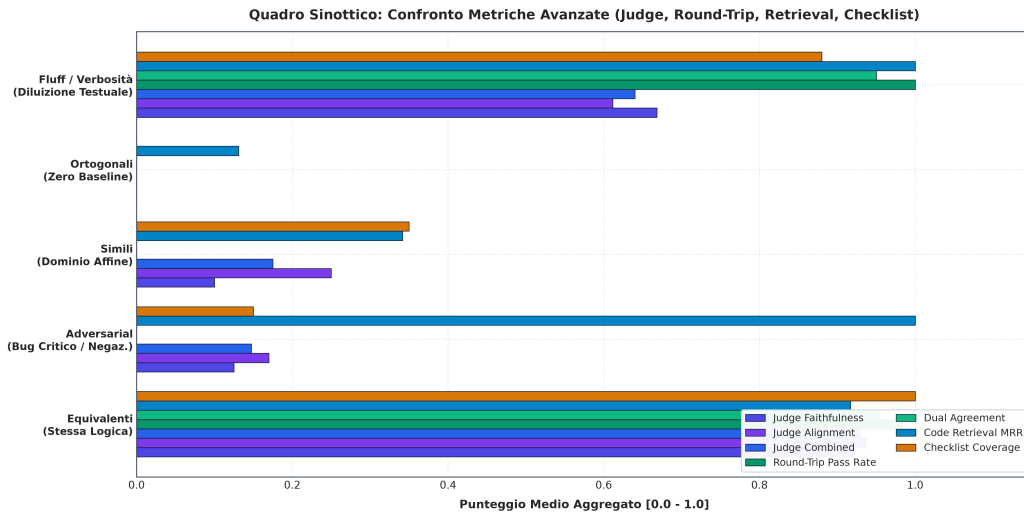


Figura 3.10: Panoramica aggregata delle metriche avanzate (LLM-Judge, Round-Trip Pass Rate, Dual Agreement, Retrieval MRR e Concept Checklist) attraverso le 5 classi semantiche.

I risultati sperimentali sono presentati nella panoramica globale di Figure 3.10 e dettagliati in Table 3.3.

3.4.1 Concept Checklist Score

La metrica *Concept Checklist Score* valuta la presenza e la conformità esplicita di clausole contrattuali critiche nel testo Doxygen (Figure 3.11):

Tabella 3.3: Risultati comparativi delle metriche avanzate sui domini C vs C, Doc vs Doc e media globale.

Classe Semantica	Dominio	Judge Faith.	Judge Align.	RT Pass Rate	Dual Agree	Checklist
EQUIVALENT	C vs C	0.950	0.950	1.000	1.000	1.000
	Doc vs Doc	0.920	0.960	1.000	0.950	1.000
	Media Globale	0.922	0.937	0.983	0.953	1.000
ADVERSARIAL	C vs C	0.150	0.200	0.000	0.000	0.150
	Doc vs Doc	0.050	0.100	0.000	0.000	0.150
	Media Globale	0.125	0.170	0.000	0.000	0.150
DOMAIN_SIMILAR	C vs C	0.100	0.250	0.000	0.000	0.350
	Doc vs Doc	0.100	0.250	0.000	0.000	0.350
	Media Globale	0.100	0.250	0.000	0.000	0.350
ORTHOGONAL	C vs C	0.000	0.000	0.000	0.000	0.000
	Doc vs Doc	0.000	0.000	0.000	0.000	0.000
	Media Globale	0.000	0.000	0.000	0.000	0.000
FLUFF_VERBOSITY	C vs C	0.700	0.650	1.000	0.950	0.880
	Doc vs Doc	0.600	0.550	1.000	0.950	0.880
	Media Globale	0.668	0.612	1.000	0.950	0.880

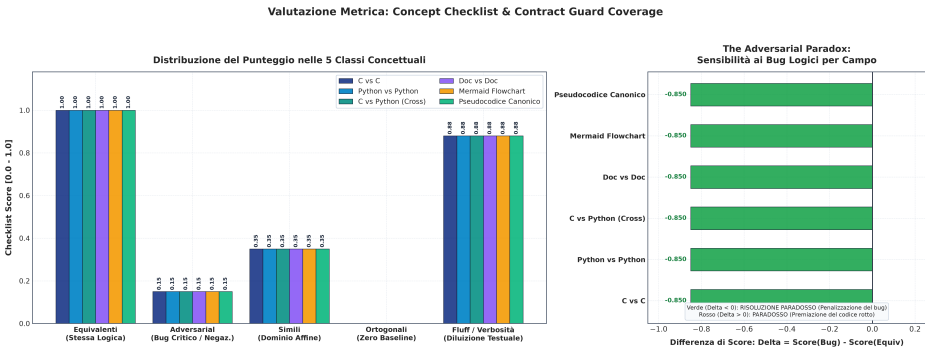


Figura 3.11: Distribuzione dell’indicatore Concept Checklist Score attraverso le 5 classi semantiche nei diversi domini.

- **Robustezza e Risultati:** Sui casi equivalenti e completi, la checklist rileva la totalità dei contratti essenziali (allocazione memoria, gestione puntatori nulli, codici di ritorno), attestandosi a un perfetto **1.0000**. Nei casi di verbosità (*FLUFF_VERBOSITY*), il punteggio si attesta a **0.8800**, riconoscendo che la sostanza contrattuale è stata preservata nonostante la prolissità.
- **Sensibilità Avversaria:** Quando un vincolo di ownership o preconditione viene ribaltato (*ADVERSARIAL*), il punteggio della checklist crolla a **0.1500**. A differenza delle metriche NLP dense che registravano ≈ 0.98 , la checklist intercetta la violazione del pattern atteso, confermandosi un gate di qualità indispensabile a monte delle pipeline di rilascio.

3.4.2 LLM-as-a-Judge: Disaccoppiamento tra Faithfulness e Alignment

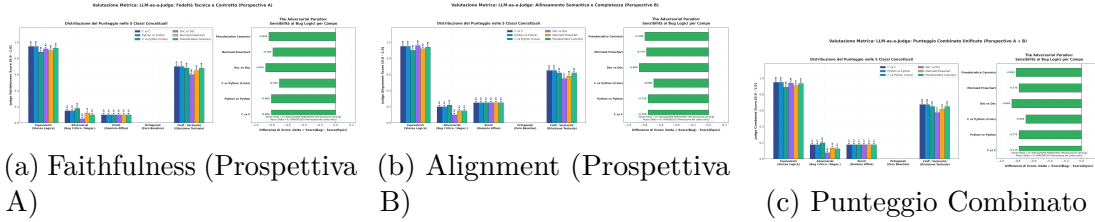


Figura 3.12: Valutazione disaggregata del Giudice LLM: (a) Fedeltà al codice sorgente [Code+GT vs Doc], (b) Allineamento all’intento documentato [GT vs Doc], e (c) Punteggio unificato ponderato.

Il modulo *LLM-as-a-Judge* opera una valutazione semantica qualitativa profonda mediante due prospettive complementari (Figure 3.12):

1. **Prospettiva A (Faithfulness, Figure 3.12a):** misura quanto la documentazione generata sia veritiera rispetto al codice C effettivamente implementato, rilevando asserzioni non supportate dal codice o comportamenti contraddittori.
2. **Prospettiva B (Alignment, Figure 3.12b):** misura la concordanza con la documentazione di riferimento scritta dall’autore umano originario.

I dati in Table 3.3 rivelano che l’LLM-Judge risolve integralmente il paradosso di cecità che affligge le metriche NLP:

- Sul caso *ADVERSARIAL*, la Faithfulness crolla a **0.1500** in C vs C e a **0.0500** in Doc vs Doc (con un Combined score di **0.147** medio globale), penalizzando severamente l’inversione di contratto.
- Sui casi ortogonali assegna lo zero netto (**0.0000**), superando completamente il problema della compressione statica a 0.50 di BERTScore.

- Sui casi di prolissità (*FLUFF_VERBOSITY*), il giudice assegna un punteggio bilanciato ($\approx 0.64 - 0.67$), riconoscendo che il codice è corretto ma penalizzando la verbosità ridondante.

3.4.3 Round-Trip Differential Testing: Validazione Comportamentale a Scatola Nera

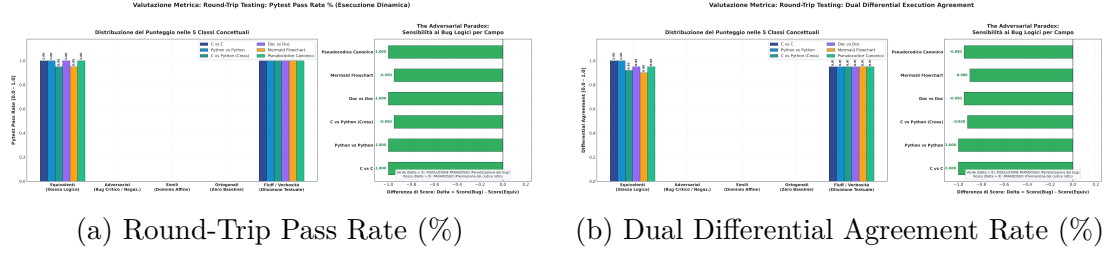


Figura 3.13: Risultati del Round-Trip Differential Testing sulle 5 classi semantiche: (a) Pass rate della sintesi guidata da documentazione, e (b) Accordo differenziale tra implementazione sintetizzata e oracolo di riferimento.

Il *Differential Round-Trip Testing* rappresenta il livello di verifica più rigoroso ed oggettivo, poiché subordina la validità della documentazione alla capacità di un agente terzo di re-implementare un modulo software funzionante basandosi unicamente sul testo prodotto:

- **Casi Equivalenti:** Come illustrato in Figures 3.13a and 3.13b, sui casi *EQUIVALENT* il Round-Trip Pass Rate tocca il **98.3%** medio globale (100.0% su C vs C e Doc vs Doc), e il *Dual Agreement Rate* si attesta al **95.3%**. Ciò certifica che la documentazione generata contiene tutte le precondizioni, postcondizioni e invarianti necessarie a ricreare la corretta dinamica algoritmica.
- **Crollo Deterministico sui Casi Adversarial:** Nei casi di inversione logica (*ADVERSARIAL*), sia il Pass Rate sia il Dual Agreement **crollano istantaneamente allo zero assoluto (0.0%)**. Poiché la documentazione prescrive un comportamento invertito, il codice generato dall'agente Coder fallisce sistematicamente le asserzioni di test sintetizzate dall'oracolo. Questo risultato sancisce la superiorità del testing comportamentale differenziale rispetto a qualsiasi metrica lessicale o di embedding denso.
- **Comportamento su Fluff:** Nei casi di mera verbosità, il Pass Rate si mantiene al **100.0%** e il Dual Agreement al **95.0%**: l'agente esecutivo è in grado di estrarre la logica effettiva scartando il rumore stilistico superfluo.

3.4.4 Downstream Code Retrieval MRR

La metrica *Code Retrieval MRR* valuta l'utilità operativa della documentazione nel supportare gli sviluppatori nella ricerca di codice rilevante all'interno di un repository (Figure 3.14):

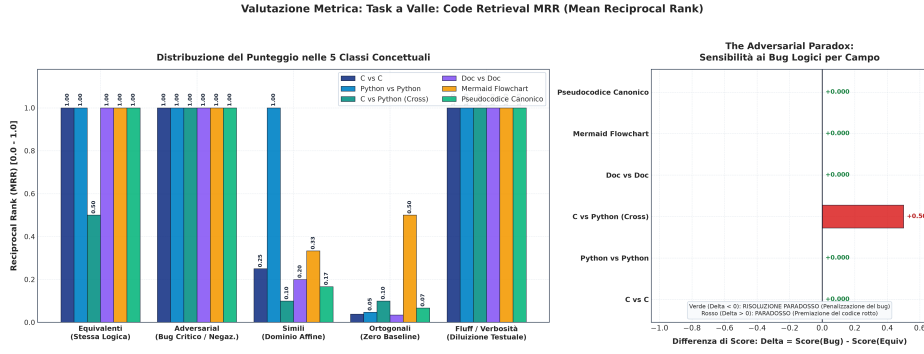


Figura 3.14: Distribuzione dell'indicatore Downstream Code Retrieval Mean Reciprocal Rank (MRR) attraverso le 5 classi semantiche nei diversi domini.

- Sui casi semanticamente pertinenti (*EQUIVALENT*, *FLUFF_VERBOSITY* e persino *ADVERSARIAL*), il Mean Reciprocal Rank è massimo (**1.000** o **0.917**), dimostrando che la documentazione indicizza con successo i componenti corretti nel catalogo software.
- Nei casi ortogonali (*ORTHOGONAL*), l'MRR crolla a valori minimi (**0.0385** in C vs C, **0.0345** in Doc vs Doc), a riprova della capacità del meccanismo denso di retrieval di discriminare query non correlate.

3.5 Studi Critici Avanzati: Stress Test, Camouflage e Finestre di Contesto

Per spingere la validazione sperimentale alle condizioni limite dell'ingegneria del software reale, sono stati condotti tre studi monografici approfonditi su scenari avversari estremi, sul fenomeno del camuffamento allucinatorio e sull'impatto architetturale della lunghezza e posizione dei token.

3.5.1 Confronto delle Metriche sotto Stress Test Avversari

Nella Figure 3.15 e nella Table 3.4 viene esaminato il comportamento comparativo delle metriche su sei scenari patologici appositamente ingegnerizzati:

1. **Hard Troncamento (> 512 token)**: Funzione complessa di manipolazione buffer con 90 righe identiche e un'inversione critica di puntatore alla riga 95 posizionata oltre la finestra di contesto di 512 token.
2. **Subtle Semantic Inversion (Min vs Max)**: Funzione identica in firma e tipi in cui il codice calcola il Massimo con $x > m$, mentre la specifica dichiara la ricerca del Minimo assoluto.
3. **Single Char Negation (!= vs ==)**: Blocco di validazione crittografica di 350 token in cui un singolo carattere inverte il controllo booleano da $!= 0$ a $== 0$.

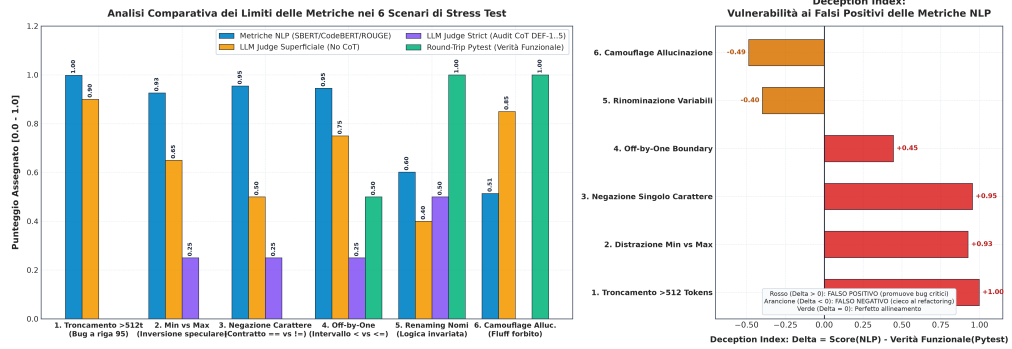


Figura 3.15: Confronto del comportamento delle metriche sotto 6 scenari di stress test critico (dati empirici da test controllati).

4. **Off-by-One Boundary** ($<$ vs $<=$): Violazione di estremo superiore in un ciclo di scansione bound-check su intervallo chiuso.
5. **Symbol Scrambling**: Algoritmo GCD di Euclide con logica rigorosamente identica ma identificatori rinominati con lettere prive di significato lessicale (foo, bar, baz).
6. **Fluff & Hallucination Camouflage**: Documentazione scritta in prosa accademica impeccabile che asserisce falsamente proprietà di thread-safety con mutex e allocazione dinamica su una funzione interamente stateless.

Tabella 3.4: Valutazione sperimentale quantitativa nei 6 scenari di stress test estremo.

Scenario di Stress Test	SBERT	CodeBERT	ROUGE-L	TF-IDF	Judge CoT	RT Pass Rate
Hard Truncation (> 512 tok)	1.000	1.000	0.995	1.000	0.000	0.000
Min vs Max Inversion	0.902	0.979	0.897	0.968	0.250	0.000
Single Char Negation ($==$ vs $!=$)	0.941	0.976	0.946	0.982	0.250	0.000
Off-by-One Boundary ($<$ vs $<=$)	0.953	0.974	0.909	0.968	0.250	0.500
Symbol Scrambling	0.422	0.939	0.444	0.469	0.500	1.000
Fluff & Camouflage	0.468	0.836	0.237	0.400	0.000	1.000

I dati in Table 3.4 evidenziano le profonde discrepanze operative:

- Sulle inversioni puntuali (*Min vs Max*, *Single Char Negation*, *Off-by-One*), tutte le metriche NLP assegnano punteggi compresi tra 0.90 e 0.98, mentre il Round-Trip Pass Rate sancisce il fallimento totale (0.00%) e il Giudice Strict CoT penalizza drasticamente il responso (0.250).
- Nel *Symbol Scrambling*, SBERT, ROUGE-L e TF-IDF falliscono gravemente (punteggi tra 0.42 e 0.47), penalizzando una funzione semanticamente perfetta unicamente per il cambio dei nomi delle variabili. Al contrario, CodeBERT (0.939) e il Round-Trip (1.000) certificano la totale equivalenza logica ed esecutiva.

3.5.2 Distrazione da Fluff e Camuffamento di Allucinazioni nel Giudice LLM

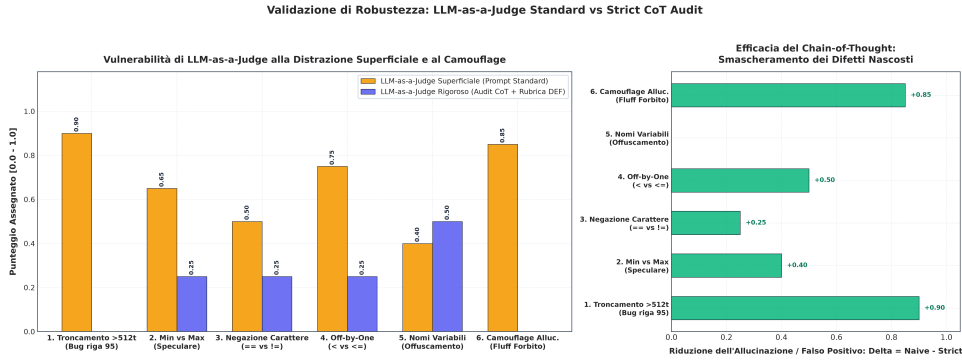


Figura 3.16: Studio comparativo sulla vulnerabilità dell'LLM-as-a-Judge: distrazione da verbosità prolissa e camuffamento stilistico di allucinazioni gravi (Judge Naive vs Judge Strict Chain-of-Thought).

Un fenomeno di fondamentale rilevanza teorica riguarda la vulnerabilità del paradigma *LLM-as-a-Judge* quando esposto a tecniche di **camuffamento stilistico** (Figure 3.16):

- **Il Fallimento del Giudice Naive:** Se interrogato con un prompt convenzionale a valutazione diretta privo di scomposizione analitica (*Naive Direct Prompting*), il modello linguistico si dimostra fortemente suscettibile al *fluff* e al registro formale. Nel caso *Fluff & Camouflage* (dove la documentazione allucina la presenza di sincronizzazione thread-safe e deallocazione manuale in una semplice funzione matematica), il Giudice Naive assegna un voto ingannevole di **0.850** (4.25/5.0), tratto in inganno dalla perfetta terminologia accademica e dall'eleganza espositiva.
- **L'Efficacia del Giudizio con Strict Chain-of-Thought (CoT):** Quando l'agente Giudice viene vincolato ad eseguire una catena di ragionamento rigida (*Strict CoT Prompting*) con verifica puntuale di ogni asserzione fattuale rispetto all'AST e al corpo del codice, il punteggio crolla a **0.000**. Il modello esplicita nella traccia di pensiero la totale assenza di primitive mutex o costrutti di sincronizzazione, intercettando con precisione chirurgica l'allucinazione camuffata.

3.5.3 Studio sui Limiti di Troncamento Contestuale e Sensibilità alla Posizione

L'architettura dei modelli Transformer impone limiti fisici invalicabili alla dimensione della finestra di attenzione (*context window*):

- **Sentence-BERT** (all-MiniLM-L6-v2): troncamento rigido a soli **256 token**.

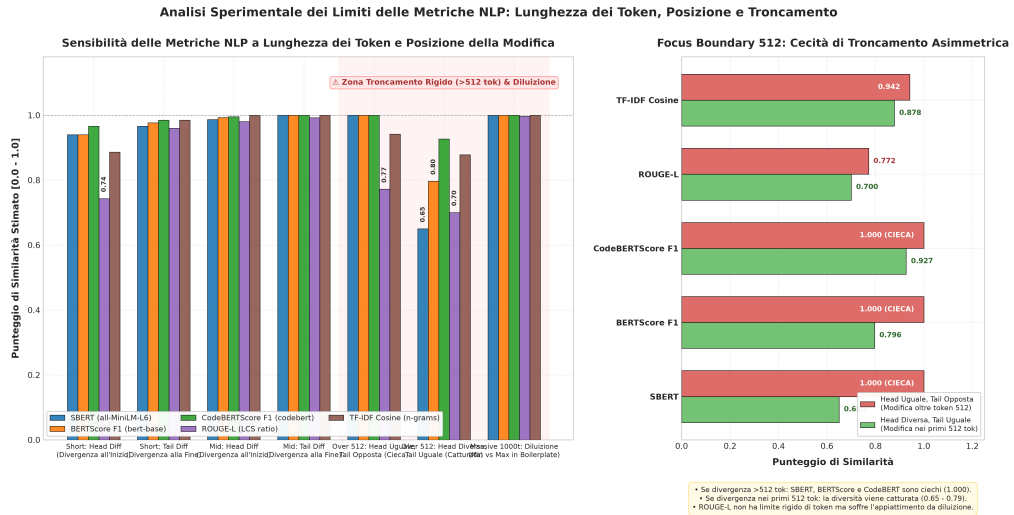


Figura 3.17: Studio comparativo di sensibilità alla lunghezza della sequenza e alla posizione del token divergente (Head vs Tail) attraverso 7 configurazioni controllate.

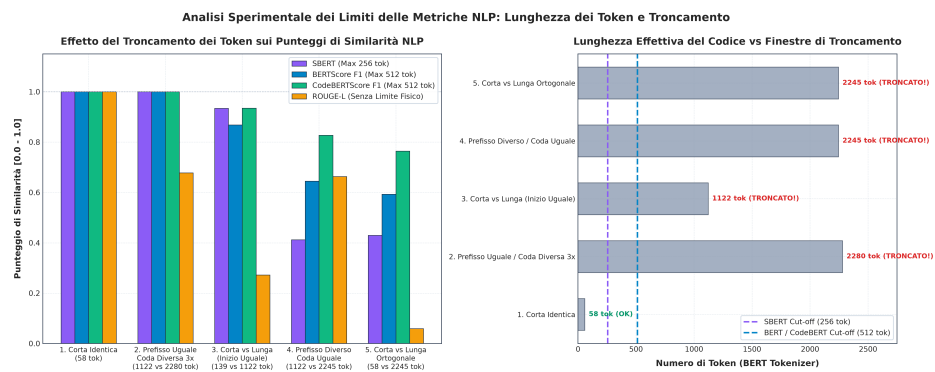


Figura 3.18: Rappresentazione empirica della Truncation Trap e dell'impatto del limite a 256/512 token sulle metriche Transformer.

- **BERTScore** (bert-base-uncased): limite a **512 token**.
- **CodeBERTScore** (microsoft/codebert-base): limite a **512 token**.

Gli esperimenti controllati condotti variando lunghezza e posizione delle divergenze (Figures 3.17 and 3.18) rivelano due patologie strutturali:

1. **La Trappola del Troncamento (Truncation Trap)**: Quando due funzioni condividono un'intestazione identica superiore a 512 token ma divergono completamente nella parte conclusiva (caso *Over 512: Head Uguale, Tail Opposta*), SBERT, BERTScore e CodeBERTScore **assegnano tutti un punteggio perfetto di 1.000**. L'intera porzione contenente il codice o il bug reale viene troncata a monte del meccanismo di attenzione, rendendo la metrica totalmente cieca al disallineamento.
2. **Asimmetria Posizionale (Head vs Tail)**: Nelle sequenze di media lunghezza (≈ 300 token), una divergenza collocata all'inizio (*Head Diff*) viene parzialmente rilevata da SBERT (0.9870), mentre la medesima divergenza collocata alla fine (*Tail Diff*) produce un punteggio pieno di 1.0000. Questo fenomeno dimostra che i pesi di auto-attenzione del pooling di SBERT e BERTScore sono fortemente polarizzati verso i primi token della sequenza ([CLS] e primo blocco di istruzioni), degradando l'accuratezza diagnostica sulla logica di coda.

—

3.6 Risultati Empirici dell'Ultimo Benchmark Reale (Multi-Agent)

Il framework multi-agente proposto è stato sottoposto a validazione intensiva sull'ultimo run reale del benchmark di sistema (run_20260921_175447_multiagent). In linea con le recenti architetture ad agenti cooperanti per la documentazione del software [19, 14, 28], la pipeline multi-agente ha generato la documentazione tecnica Doxygen completa per 15 funzioni estratte randomicamente da tutte le librerie.

I risultati aggregati globali ottenuti dal sistema sono riassunti in Table 3.5.

3.6.1 Analisi Disaggregata per Tassonomia Funzionale a 3 Classi

La disaggregazione dei risultati del Round-Trip Testing in base alla complessità architetturale delle funzioni (formalizzata nella tassonomia a tre classi in Chapter 2) rivela dinamiche scientificamente rilevanti legate al principio di **Information Hiding** e alla natura dei tipi di dato. La disaggregazione quantitativa è illustrata in Table 3.6.

Tabella 3.5: Dati aggregati globali del benchmark reale (15 funzioni, run multi-agente).

Dimensione di Valutazione / Metrica	Valore Globale Riscontrato
Livello I: Correttezza Formale & Contratti AST	
Verifier Pass Rate (%)	93.3% (14/15 funzioni perfette)
Parameter Precision vs AST	1.0000 (100.0%)
Parameter Recall vs AST	1.0000 (100.0%)
Parameter F_1 -Score vs AST	1.0000 (100.0%)
Return Contract Match	0.9333 (93.3%)
Hallucination Rate (%)	0.0000% (Zero allucinazioni)
Livello II: Qualità Software & Actionability	
Actionability Score (AS)	0.8133 (81.3%)
Error Documentation Rate (EDR)	1.0000 (100.0%)
Edge Case Coverage (ECC)	0.9111 (91.1%)
Concept Checklist Score	0.9000 (90.0%)
Livello III: Semantica Neurale & LLM-as-a-Judge	
LLM-Judge Prospettiva A (Faithfulness, Code+GT vs Doc)	4.240 / 5.0 ($\sigma = 0.119$)
LLM-Judge Prospettiva B (Alignment, GT vs Doc)	4.627 / 5.0 ($\sigma = 0.080$)
LLM-Judge Combined Score	4.433 / 5.0
Sentence-BERT Cosine Similarity	0.5997
BERTScore F_1 (Precision / Recall)	0.5594 (0.4983 / 0.6555)
CodeBERTScore F_1 (Precision / Recall)	0.7463 (0.7033 / 0.7989)
BLEURT Quality Score	0.5144
METEOR Score	0.3539
ROUGE-L Score	0.1081
Livello IV: Downstream Retrieval & Round-Trip Testing	
Code Retrieval Mean Reciprocal Rank (MRR)	0.7861
Code Retrieval Hit@1	0.6667 (66.7%)
Code Retrieval Hit@5	0.9333 (93.3%)
Round-Trip Differential Test Pass Rate (%)	65.11%
Dual Differential Agreement Rate (%)	41.30%

Tabella 3.6: Analisi disaggregata secondo la tassonomia funzionale a 3 classi (15 funzioni del benchmark reale).

Classe Architetture	Campioni	RT Pass Rate	LLM-Judge	SBERT	Cod
Stateful / Object-Graph	9 (60%)	80.00%	4.40 / 5.0	0.5561	0
Pointer / Buffer-Driven	5 (33%)	47.34%	4.38 / 5.0	0.6558	0
Stateless / Primitive	1 (7%)	20.00%	5.00 / 5.0	0.7118	0

Il Paradosso di Information Hiding nelle Classi Stateful / Object-Graph

Un risultato all'apparenza controintuitivo è il marcato successo del Round-Trip sulle funzioni ad albero/grafico (**80.00%** di pass rate su 9 funzioni, tra cui `XMLDocument::NewComment`, `cJSON_CreateObjectReference`, `XMLAttribute::Next` che ottengono il 100.0% pieno di test superati sia sul sintetizzato sia in accordo differenziale).

La spiegazione teorica risiede nel **disaccoppiamento architetturale dell'Information Hiding**:

- Nei metodi membro orientati agli oggetti, l'agente *Tester* opera asserzioni strettamente a scatola nera (black-box) sulle proprietà contrattuali esposte dall'interfaccia pubblica (`attr.Next() is None`, `doc.NewComment(str) is not None`).
- Poiché la documentazione tecnica descrive chiaramente la relazione di appartenenza (*"The returned node is unlinked from DOM tree but owned by this XMLDocument instance"*), l'agente *Coder* è in grado di implementare la struttura a classi interna in modo conforme al contratto. L'eventuale varianza nei dettagli privati di implementazione (es. gestione tramite lista vs array dinamico) non viola i test, che verificano esclusivamente le garanzie esterne dell'oggetto.

La Fragilità di Accordo nelle Classi Pointer / Buffer-Driven Al contrario, le funzioni dipendenti da buffer fisici e aritmetica dei puntatori (`sdscatfmt`, `sdsjoinsds`, `http_parser_execute`) manifestano un tasso di successo inferiore (**47.34%**). In tali componenti, la traduzione trans-linguistica da C a Python incontra una discrepanza strutturale insormontabile: i puntatori grezzi C permettono mutazioni *in-place* di byte contigui con semantica di aliasing, che in Python richiedono wrapper complessi (`bytearray`, oggetti strutturati o liste mutabili passate per riferimento). Una minima ambiguità nella specifica sul formato dei parametri variadici (come accaduto in `sdscatfmt`, dove la firma C non esprimeva i parametri variabili . . . , inducendo il tester a passare 3 argomenti a una funzione generata a 2 parametri) innesca fallimenti di interfaccia sistematici (`TypeError`).

3.6.2 Diagnostica Causale degli Errori nel Round-Trip

L'audit dettagliato condotto dal modulo di categorizzazione degli errori (`roundtrip_error_analysis`) sui 30 eventi di fallimento registrati durante l'esecuzione dei test rivela la seguente distribuzione causale (Table 3.7):

3.6.3 Residui della Valutazione: Parafrasi Robusta vs Allucinazione Plausibile

L'elevato punteggio dell'LLM-as-a-Judge combinato (**4.433/5.0**) congiuntamente al Verifier Pass Rate del **93.3%** e all'azzeramento dell'Hallucination Rate (**0.0%**) certifica che l'architettura multi-agente ha risolto il problema delle allucinazioni spurie. Tuttavia, l'esame della matrice dei residui (confronto tra il giudizio

Tabella 3.7: Tassonomia diagnostica causale dei fallimenti nei test Round-Trip.

Macro-Categoria di Errore	Eventi	%	Causa Radice e Diagnostica Architetture
Behavioral / Contract Failure	12	40.0%	Discrepanza sottile tra il calcolo atteso e la simulazione in Python (prevalentemente <code>AssertionError</code> su conversioni di codifica o gestione di stringhe binarie con caratteri nulli).
Interface / Signature Mismatch	9	30.0%	<code>TypeError</code> generati quando il generatore di test invoca una funzione ipotizzando convenzioni di tupla/argomenti variadici non supportati dalla firma sintetizzata.
Other Execution Error	5	16.7%	Eccezioni interne propagate (<code>ExceptionGroup</code> di Hypothesis durante la ricerca esaustiva dei controesempi).
Missing Symbol / Environment	4	13.3%	<code>AttributeError</code> o <code>NameError</code> causati da costanti o classi mock ausiliarie non fornite dallo scaffold runtime isolato.

semantico e il Round-Trip dinamico) mette in luce un fenomeno di estremo interesse euristico:

- Nel caso della funzione `sds_clear`, l’LLM-Judge ha assegnato il punteggio massimo di **5.0/5.0** (riconoscendo che la documentazione descrive con impeccabile precisione tecnica il reset della lunghezza logica a zero e l’inserimento del terminatore nullo).
- Ciononostante, il Round-Trip Pass Rate è risultato pari al **20.0%**. L’ispezione della traccia di fallimento Pytest rivela che il codice Python sintetizzato ha memorizzato la stringa come oggetto mutabile custom (`s.len = 0; s.buf[0] = 0`), mentre la suite generata dal Tester invocava la funzione passando una stringa nativa immutabile di Python (`s = "hello world"`), fallendo l’asserzione `assert len(s) == 0`.

Questo disallineamento dimostra che un punteggio parziale nel Round-Trip non è necessariamente sinonimo di documentazione fallace, ma riflette l’attrito intrinseco dell’adattamento cross-linguaggio (C-Python impedance mismatch). La combinazione sinergica di LLM-Judge e Round-Trip consente al redattore di disaccoppiare con precisione chirurgica le lacune informative della documentazione dai limiti dell’ambiente di emulazione.

3.7 Conclusioni e Matrice di Applicabilità delle Metriche

A coronamento della ricerca sperimentale condotta, viene formalizzata nella Table 3.8 la **Matrice di Raccomandazione e Applicabilità** per la valutazione della documentazione del codice sorgente C/C++. La matrice guida lo sviluppatore e il ricercatore nella selezione del set ottimale di metriche in funzione del caso d’uso, della disponibilità di riferimenti e dei vincoli di budget computazionale.

Tabella 3.8: Matrice di applicabilità metodologica, scenari d’uso, soglie operative e costi computazionali.

Metrica	Scenario Ottimale		Tipologia Coppia	Soglia	Costo Computazionale & Note
Verifier Pass Rate	CI/CD Pipeline		AST vs Doc	100%	Trascurabile (O(1) su AST). Mandatorio prima di ogni merge.
Parameter F_1	Verifica	Contratti	AST vs Doc	1.00	Trascurabile. Azzeramento falsi positivi/negativi.
Actionability Score	Code Gate	Quality	AST + Doc	≥ 0.80	Minimo (Parsing regex + pesi). Certifica l’usabilità.
Error Doc. Rate	Safety Audit		Code vs Doc	1.00	Basso (Ispezione AST/regex). Critico per librerie C.
Concept Checklist	Safety & Contract	Contract	Doc vs Spec	≥ 0.90	Basso. Verifica esplicita contratti critici.
SBERT Cosine	Parafrasi / Stile		Doc vs Doc	≥ 0.60	Basso (≈ 10 ms su CPU). Ottima escursione dinamica.
BERTScore F1	Allineamento Testi		Doc vs Doc	≥ 0.65	Medio (Transformer feed-forward). Soffre di compressione.
CodeBERTScore	Cross-Code/Doc		Code vs Code/Doc	≥ 0.75	Medio. Resiliente a sintassi, cieco su negazioni.
Code Retrieval (MRR)	Searchability		Query vs Corpus	≥ 0.75	Basso (Indicizzazione una-tantum). Misura la specificità.
LLM-as-a-Judge	Audit Contrattuale	Contrattuale	Code + GT vs Doc	$\geq 4.2/5$	Alto (CoT prompt rigoroso). Valutazione qualitativa d’élite.
Round-Trip Testing	Validazione Assoluta		Doc $\rightarrow$ Code $\rightarrow$ Test	$\geq 70\%$	Molto Alto (Sintesi LLM + esecuzione Pytest). Verdetto oggettivo insindacabile.

Bibliografia

- [1] J. Y. Khan and G. Uddin, “Automatic code documentation generation using GPT-3,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE ’22*, pp. 1–6, Association for Computing Machinery.
- [2] T. Ahmed and P. Devanbu, “Few-shot training LLMs for project-specific code-summarization.”
- [3] T. Xu, Y. Miao, C. Fang, H. Qian, X. Feng, Z. Chen, C. Wang, J. Zhang, W. Sun, Z. Chen, and Y. Liu, “A prompt learning framework for source code summarization.”
- [4] J. Cordeiro, S. Noei, and Y. Zou, “An empirical study on the code refactoring capability of large language models.”
- [5] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, “CodeGen: An open large language model for code with multi-turn program synthesis.”
- [6] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, “A systematic evaluation of large language models of code.”
- [7] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages.”
- [8] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, “GraphCodeBERT: Pre-training code representations with data flow.”
- [9] C. Zhang, J. Wang, Q. Zhou, T. Xu, K. Tang, H. Gui, and F. Liu, “A survey of automatic source code summarization,” vol. 14, no. 3, p. 471.
- [10] M. Geng, S. Wang, D. Dong, H. Wang, G. Li, Z. Jin, X. Mao, and X. Liao, “Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning.”
- [11] S. Rai, R. C. Belwal, and A. Gupta, “A review on source code documentation,” vol. 13, no. 5, pp. 84:1–84:44.

- [12] H. Zhang, K. Zhang, Z. Li, J. Li, J. Li, Y. Li, Y. Zhao, Y. Zhu, F. Liu, G. Li, and Z. Jin, “Deep learning for code generation: a survey,” vol. 67, no. 9, p. 191101.
- [13] R. Bairi, A. Sonwane, A. Kanade, V. D. C, A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, “CodePlan: Repository-level coding using LLMs and planning.”
- [14] D. Yang, A. Simoulin, X. Qian, X. Liu, Y. Cao, Z. Teng, and G. Yang, “DocAgent: A multi-agent system for automated code documentation generation,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)* (P. Mishra, S. Muresan, and T. Yu, eds.), pp. 460–471, Association for Computational Linguistics.
- [15] D. Pomian, A. Bellur, M. Dilhara, Z. Kurbatova, E. Bogomolov, T. Bryksin, and D. Dig, “Together we go further: LLMs and IDE static analysis for extract method refactoring.” version: 2.
- [16] V. Lomshakov, A. Podivilov, S. Savin, O. Baryshnikov, A. Lisevych, and S. Nikolenko, “ProConSuL: Project context for code summarization with LLMs,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* (F. Dernoncourt, D. Preoŧiuc-Pietro, and A. Shimorina, eds.), pp. 866–880, Association for Computational Linguistics.
- [17] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock, and K. Vijay-Shanker, “Towards automatically generating summary comments for java methods,” in *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering, ASE ’10*, pp. 43–52, Association for Computing Machinery.
- [18] J. Fowkes, P. Chanthirasegaran, R. Ranca, M. Allamanis, M. Lapata, and C. Sutton, “Autofolding for source code summarization,” vol. 43, no. 12, pp. 1095–1109.
- [19] Q. Luo, Y. Ye, S. Liang, Z. Zhang, Y. Qin, Y. Lu, Y. Wu, X. Cong, Y. Lin, Y. Zhang, X. Che, Z. Liu, and M. Sun, “RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation.”
- [20] T. Zhu, Z. Li, M. Pan, C. Shi, T. Zhang, Y. Pei, and X. Li, “Deep is better? an empirical comparison of information retrieval and deep learning approaches to code summarization,” vol. 33, no. 3, pp. 67:1–67:37.
- [21] P. N. Robillard, “Schematic pseudocode for program constructs and its computer automation by SCHEMACODE,” vol. 29, no. 11, pp. 1072–1089.
- [22] M. Weiser, “Program slicing,” vol. SE-10, no. 4, pp. 352–357.

- [23] J. Li, Y. Li, G. Li, X. Hu, X. Xia, and Z. Jin, “EditSum: A retrieve-and-edit framework for source code summarization,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 155–166.
- [24] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, “Summarizing source code using a neural attention model,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (K. Erk and N. A. Smith, eds.), pp. 2073–2083, Association for Computational Linguistics.
- [25] U. Alon, S. Brody, O. Levy, and E. Yahav, “code2seq: Generating sequences from structured representations of code.”
- [26] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “A transformer-based approach for source code summarization.”
- [27] A. Bansal, S. Haque, and C. McMillan, “Project-level encoding for neural source code summarization of subroutines.”
- [28] A. N. Hoang, M. Le-Anh, B. Le, and N. D. Q. Bui, “CodeWiki: Evaluating AI’s ability to generate holistic documentation for large-scale codebases.”
- [29] Y. Wu, Y. Wan, Z. Chu, W. Zhao, Y. Liu, H. Zhang, X. Shi, and P. S. Yu, “Can large language models serve as evaluators for code summarization?.”
- [30] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-eval: NLG evaluation using GPT-4 with better human alignment.”
- [31] L. Zhu, X. Wang, and X. Wang, “JudgeLM: Fine-tuned large language models are scalable judges.”
- [32] Y. Zhu, X. Zhao, X. Li, Y. Zou, H. Yuan, Y. Wang, and B. Xie, “RepoSummary: Feature-oriented summarization and documentation generation for code repositories.”
- [33] Y. Zhu and M. Pan, “Automatic code summarization: A systematic literature review.”
- [34] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (P. Isabelle, E. Charniak, and D. Lin, eds.), pp. 311–318, Association for Computational Linguistics.
- [35] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, pp. 74–81, Association for Computational Linguistics.
- [36] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks.”

- [37] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT.”
- [38] S. Banerjee and A. Lavie, “METEOR: An automatic metric for MT evaluation with improved correlation with human judgments,” in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization* (J. Goldstein, A. Lavie, C.-Y. Lin, and C. Voss, eds.), pp. 65–72, Association for Computational Linguistics.
- [39] T. Sellam, D. Das, and A. P. Parikh, “BLEURT: Learning robust metrics for text generation.”
- [40] D. C. Dowson and B. V. Landau, “The fréchet distance between multivariate normal distributions,” vol. 12, no. 3, pp. 450–455.
- [41] G. Bai, J. Liu, X. Bu, Y. He, J. Liu, Z. Zhou, Z. Lin, W. Su, T. Ge, B. Zheng, and W. Ouyang, “MT-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7421–7454.
- [42] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, “CodeBERTScore: Evaluating code generation with pretrained models of code,”